

Swift: Fast Secure Neural Network Inference With Fully Homomorphic Encryption

Yu Fu¹, Yu Tong, Yijing Ning¹, Tianshi Xu¹, Meng Li¹, *Member, IEEE*,
Jingqiang Lin¹, *Senior Member, IEEE*, and Dengguo Feng²

Abstract—With the widespread use of machine learning (ML), privacy concerns during neural network inference are attracting growing attention. Secure two-party neural network (2PC-NN) inference is the privacy-preserving inference method, which allows client to obtain the inference result without disclosing client's input to the server. The server's model parameters are also confidential to the client. However, current 2PC-NN inference schemes still have large overhead, especially for non-linear functions. In this paper, we present Swift, a fast secure 2PC-NN inference scheme based on fully homomorphic encryption (FHE) and secret sharing (SS). FHE protects the input and model parameters in linear functions, while SS is integrated to protect the non-linear functions. Concretely, Swift integrates FHE and SS to design secure and efficient non-linear protocols used for ReLU and max pooling. To further optimize performance, Swift employs FHE with computation-friendly coefficient encoding for fast execution of linear functions, and SIMD encoding for non-linear functions. Swift constructs efficient encoding conversion protocol between the coefficient-encoded ciphertext and the SIMD-encoded ciphertext. Finally, Swift achieves secure neural network inference framework for MNIST dataset. Compared with Cheetah (USENIX 2022), the execution time of ReLU, max pooling, secure inference under a WAN setting improves 7.4×, 13.3×, 1.9×, respectively.

Index Terms—Neural network inference, privacy-preserving inference, fully homomorphic encryption.

I INTRODUCTION

IN RECENT years, ML has been applied in various applications such as medical diagnosis, biometric authentication and credit-risk assessment [1]. Many technology companies begin to offer machine learning as a Service (MLaaS), which helps clients benefit from ML without training the neural network model by themselves. The client uploads the input

data, the server employs the pre-trained model to perform inference and returns the inference result. However, this scenario inevitably raises privacy concerns. On the one hand, clients may not want to disclose their sensitive data to the server. On the other hand, the server is also unwilling to disclose the model parameters to clients. Since the trained model has significant commercial value, the server protects the model so that clients cannot extract sensitive information.

To alleviate privacy concerns, many works [2], [3], [4], [5], [6], [7], [8], [9], [10], [11], [12], [13], [14], [15], [16], [17], [18], [19], [20], [21], [22] have constructed secure neural network inference schemes based on homomorphic encryption (HE) and multi-party computation (MPC). The proposed schemes ensure clients' input and servers' model will not be revealed except themselves. Concretely, cryptographic protocols are proposed to implement linear and non-linear functions in a neural network. FHE is the cryptographic primitive for the protocols of linear functions. Initially, researchers employ FHE with SIMD (Single Instruction Multiple Data) encoding [23] for batch processing. SIMD encoding can amortize the cost of homomorphic operations. However, rotation in SIMD-tailored protocols [7], [9], [10], [11] is expensive. Further, Cheetah [13] proposes new computation-friendly encoding method for convolution and fully connection, which avoids time-consuming rotation. With Cheetah's protocol, single homomorphic multiplication directly obtains the result of a linear function. In this paper, we denote the encoding method in Cheetah [13] as coefficient encoding.

Although protocols for linear functions are practical enough, non-linear protocols still have performance bottlenecks. As described in Section II-A, word-wise FHE is difficult to handle non-linear functions. Most works [4], [6], [7], [8] describe non-linear functions (e.g. ReLU) as boolean circuits, and evaluate them through garbled circuit (GC) [24]. However, GC incurs significant communication overhead. Non-linear protocols [12], [13] based on oblivious transfer (OT) perform better than GC-based protocols, but require multi-round interactions.

Our work proposes a new secure 2PC-NN inference scheme with FHE and SS, called Swift, aiming to achieve better performance. Swift employs FHE to protect the input and model parameters in linear functions, and employs coefficient encoding for fast execution of linear functions. As for non-linear functions, SS is integrated with FHE to perform secure computation of ReLU and max pooling. SIMD encoding is employed to calculate our proposed non-linear protocols. Further more, Swift designs efficient conversion pro-

Received 10 June 2024; revised 18 December 2024; accepted 20 February 2025. Date of publication 3 March 2025; date of current version 12 March 2025. This work was supported by the National Key Research and Development Program of China under Grant 2022YFB4501500. The associate editor coordinating the review of this article and approving it for publication was Dr. Jun Zhao. (*Corresponding author: Jingqiang Lin.*)

Yu Fu, Yu Tong, Yijing Ning, and Jingqiang Lin are with the School of Cyber Science and Technology and Anhui Province Key Laboratory of Digital Security, University of Science and Technology of China, Hefei, Anhui 230027, China (e-mail: fuyu22@mail.ustc.edu.cn; ustc-tongyu@mail.ustc.edu.cn; truegeorge@mail.ustc.edu.cn; linjq@ustc.edu.cn).

Tianshi Xu is with the School of Integrated Circuits, Peking University, Beijing 100871, China (e-mail: tianshixu@stu.pku.edu.cn).

Meng Li is with the Institute for Artificial Intelligence and the School of Integrated Circuits, Peking University, Beijing 100871, China (e-mail: meng.li@pku.edu.cn).

Dengguo Feng is with the Institute of Software, Chinese Academy of Sciences, Beijing 100190, China (e-mail: fengdg@263.net).

Digital Object Identifier 10.1109/TIFS.2025.3547308

tol between coefficient encoding and SIMD encoding. Overall, the main contributions of this paper are three-fold:

- 1) Different from current MPC-based non-linear protocols, Swift integrates FHE with SS to construct more efficient ReLU and max pooling protocols. Compared with Cheetah, the execution time of our ReLU protocol improves $7.2\times$ under a LAN setting, and improves $7.4\times$ under a WAN setting. Max pooling improves $4.5\times$ under a LAN setting, and improves $13.3\times$ under a WAN setting.
- 2) Swift selects the optimal FHE encoding methods for linear and non-linear functions respectively (e.g. coefficient encoding for convolution and fully connection, SIMD encoding for our proposed ReLU), also designs efficient encoding conversion protocol between the coefficient-encoded ciphertext and the SIMD-encoded ciphertext.
- 3) Swift implements the secure 2PC-NN inference for MNIST dataset. Compared with Cheetah, the execution time improves $1.7\times$ under a LAN setting, and improves $1.9\times$ under a WAN setting.

The rest of the paper is organized as follows. Section II briefly describes the related work. Section III introduces the preliminary. Section IV presents the details of our Swift framework and Section V evaluates the performance. Section VI concludes this paper.

II RELATED WORK

In this section, we first describe FHE-based computation for non-linear functions, then present some typical two-party and multi-party secure neural network inference schemes.

A. FHE-Based Computation for Non-Linear Functions

Word-wise FHE schemes BFV [25], BGV [26] and CKKS [27] can easily calculate convolution and fully connection, but they are difficult to handle non-linear ReLU and max pooling. One solution is to convert word-wise FHE into bit-wise FHE (TFHE [28] and FHEW [29]). However, existing conversion schemes [30], [31] incur significant costs. As a compromise, some schemes choose to avoid non-linear ReLU and max pooling. For example, CryptoNets [2] recommends that max pooling is substituted by scaled mean pooling. [32] utilizes computation-friendly polynomials to replace ReLU. In nGraph-HE2 [10], the client decrypts the encrypted data and uses plaintext to calculate ReLU. Obviously, nGraph-HE2 leaks the information about server's model to clients. To protect the privacy, Intel's Software Guard Extensions (SGX) [33] is suggested to be employed by nGraph-HE2. However, SGX faces the risk of side channel attacks [34], [35], [36], [37], [38].

B. Secure Neural Network Inference

CryptoNets [2] is the first FHE-based 2PC-NN inference scheme. The following works [3], [9], [10] aim to improve performance and accuracy. Most secure neural network inference schemes [4], [5], [6], [7], [8], [12], [13] integrate different cryptographic primitives. They employ HE and SS for linear

functions, GC and OT for non-linear functions. DeepSecure [14] and XONN [15] are designed for binary neural networks. GC and OT can be employed for calculating both linear functions and non-linear functions.

Some works consider the secure outsourced computation scenario. The client employs SS to split input into shares, then sends the shares of input to a set of parties. Each party holds a share of the trained model. The set of parties collaboratively perform secure neural network inference, then return the shares of inference result. According to the number of parties, there are schemes for three parties [16], [17], [18], [19], four parties [20], [21] and more parties [22]. These schemes have constructed more efficient linear and non-linear protocols than two-party schemes while they require more participants.

III PRELIMINARIES

In this section, we first introduce the FHE scheme we adopted, CKKS. Next, we present the FHE-based computation with different encoding methods. Specifically, we present the computation with SIMD encoding and coefficient encoding, respectively.

A. Fully Homomorphic Encryption

Most current FHE schemes are based on learning-with-errors (LWE) or its ring variant, ring learning-with-errors (RLWE). Among them, CKKS supports approximate arithmetic operations over real or complex numbers, and is frequently adopted by secure neural network inference schemes [32], [39], [40], [41]. Swift also employs CKKS. We use $\mathbb{Z}_q[x]$ to denote the ring of polynomials with coefficients in $\mathbb{Z}_q$. $\mathbb{Z}_q = \mathbb{Z} \cap (-q/2, q/2]$, $\mathbb{Z}$ is the set of integer numbers and q is the coefficient modulus. The polynomial quotient ring used for CKKS is $\mathbb{R}_q = \mathbb{Z}_q[x]/(x^n + 1)$ that equals $\mathbb{Z}_q[x]$ modulo $(x^n + 1)$, n is a power of 2. CKKS parameters are (n, q, δ) . Secret key is $sk = s \in \mathbb{R}_q$. Public key is $pk = (p_0, p_1) = (-p_1 \cdot s + e, p_1)$, $p_1 \in \mathbb{R}_q$ is chosen uniformly at random, the error e and the following $e_0, e_1 \in \mathbb{R}_q$ are sampled from a discrete Gaussian distribution of the small variance δ^2 . When encrypting a message m , the corresponding computation is $\text{Enc}(m) = (c_0, c_1) = (m + u \cdot p_0 + e_0, u \cdot p_1 + e_1)$, and $u \in \mathbb{R}_q$ is sampled from $\{0, \pm 1\}$ uniformly. When decrypting the ciphertext $\text{Enc}(m)$, the corresponding computation is $\text{Dec}(c_0, c_1) = c_0 + c_1 \cdot s = m$.

The original CKKS scheme [27] requires large coefficient modulus. To avoid multi-precision computation, a residue number system variant of CKKS called RNS-CKKS [42] is proposed. RNS-CKKS employs SIMD encoding by default, and is supported by mainstream FHE libraries, such as Microsoft SEAL [43]. In the residue number system, a large modulus is decomposed into several relative small moduli by using the Chinese remainder theorem. Therefore, operations on large modulus are equivalent to the computation on corresponding small moduli.

FHE schemes support homomorphic addition ($\boxplus$), homomorphic subtraction ($\boxminus$) and homomorphic multiplication ($\boxtimes$). Given plaintexts PT_0, PT_1 , and their ciphertexts CT_0, CT_1 , the result of $CT_0 \boxplus CT_1$ (resp. $CT_0 \boxminus CT_1$) is the ciphertext of

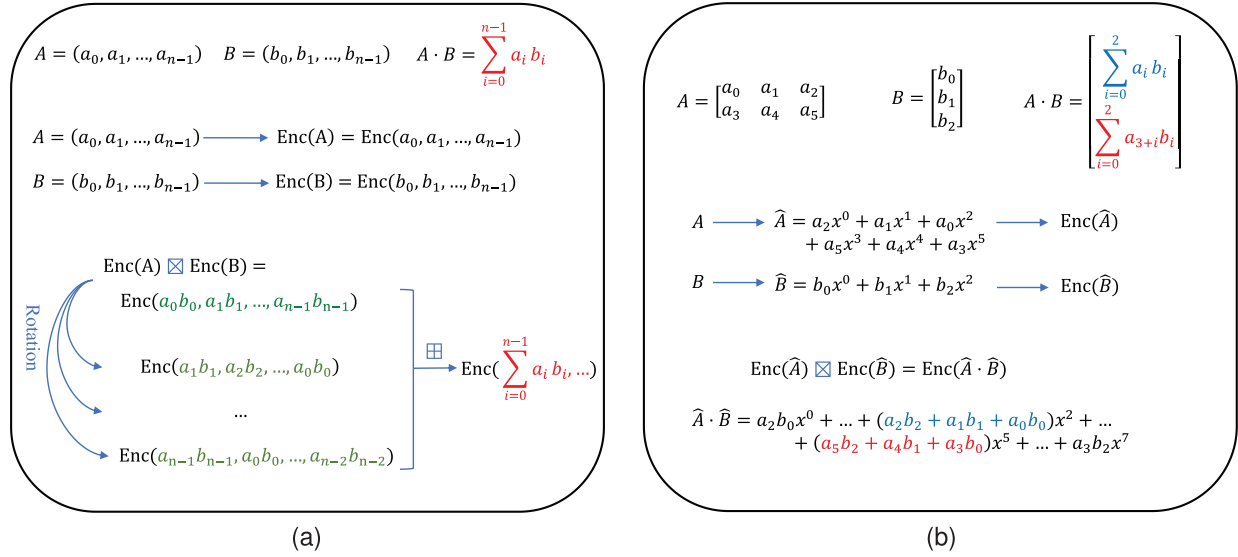


Fig. 1. FHE-based computation with different encoding. (a) SIMD encoding. (b) Coefficient encoding.

$\text{PT}_0 + \text{PT}_1$ (resp. $\text{PT}_0 - \text{PT}_1$). The result of $\text{CT}_0 \boxtimes \text{CT}_1$ is the ciphertext of $\text{PT}_0 \cdot \text{PT}_1$.

B. FHE-Based Computation With Different Encoding

Convolution and fully connection are linear functions in a neural network. They can be regarded as a batch of inner products. In FHE-based computation, two employed encoding methods are SIMD encoding and coefficient encoding.

1) *SIMD Encoding*: Since the initial FHE scheme [44] has low computation efficiency, batch processing is supposed to be employed to amortize the cost of homomorphic operations. The concept of SIMD is first proposed in [45], which uses a single instruction to operate multiple data simultaneously. In the field of FHE, SIMD encoding [23] is proposed for handling homomorphic operations in parallel. SIMD encoding converts vectors into polynomials, the addition and multiplication of polynomials are equivalent to element-wise addition and element-wise multiplication of vectors.

The procedure of calculating an inner product with SIMD encoding is presented in Figure 1a. A and B are encoded, then are encrypted as $\text{Enc}(A)$ and $\text{Enc}(B)$. Homomorphic multiplication in ciphertexts represents the element-wise multiplication between A and B , yielding a single ciphertext $\text{Enc}(a_0b_0, a_1b_1, \dots, a_{n-1}b_{n-1})$. Next, we need to add all the elements $a_i b_i, i \in [0, n-1]$. Rotation operations are performed. After homomorphic addition of the rotated ciphertexts, we get the result of the inner product. This computation method is straightforward, but rotation is rather expensive.

2) *Coefficient Encoding*: Compared with SIMD encoding, coefficient encoding avoids expensive rotation, achieving better computation performance. Coefficient encoding places elements from input and weight parameters onto polynomial coefficients in a specific way. After polynomial multiplication, some coefficients in the product are directly the result of fully connection or convolution.

We take fully connection as the example to illustrate encoding details. The size of weight matrix A is $n_l \times n_k$, the input

is a vector B of length n_k . The coefficients in encoded A are organized as $\hat{A}[i \cdot n_k + n_k - 1 - j] = A[i, j], i \in [0, n_l - 1]$ and $j \in [0, n_k - 1]$. The coefficients in encoded B are organized as $\hat{B}[j] = B[j], j \in [0, n_k - 1]$. The $(i \cdot n_k + n_k - 1)$ -th coefficients in $\hat{A} \cdot \hat{B}$ for all $i \in [0, n_l - 1]$ are exactly the result of $A \cdot B$. The computation process is presented in Figure 1b when $n_l = 2$ and $n_k = 3$. After encoding and encryption, one homomorphic multiplication directly calculates the result of the fully connection. For more details, please refer to [13].

IV THE SWIFT FRAMEWORK

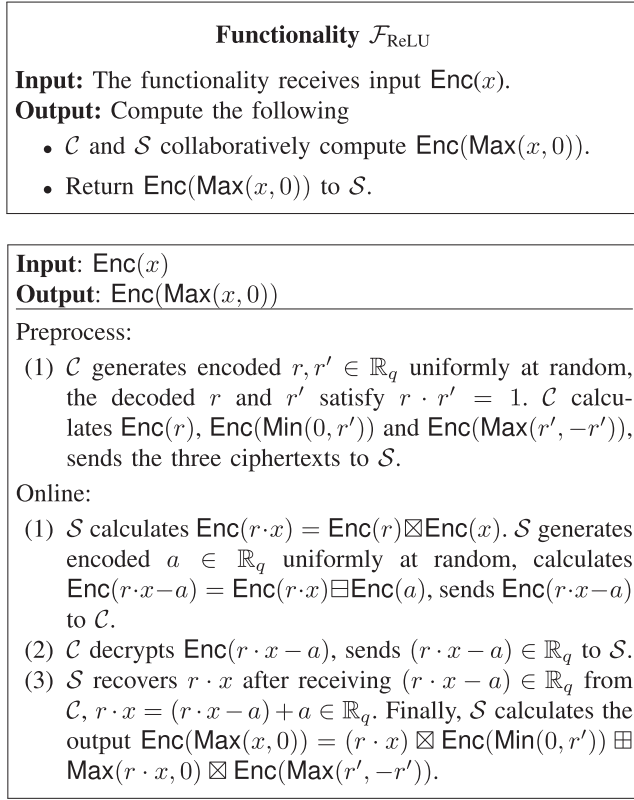
In this section, we first introduce the threat model, then present our protocols for ReLU and max pooling. We also present the Swift secure neural network inference framework. Since FHE can directly calculate linear functions, we do not present details here.

A. Threat Model

Swift works for the two-party semi-honest setting. The client $\mathcal{C}$ holds the private input, and the server $\mathcal{S}$ holds the model. During the secure 2PC-NN inference, $\mathcal{C}$ and $\mathcal{S}$ follow the protocol, and only one of them is corrupted by the adversary. The corrupted party tries to learn additional information from received messages. Swift targets to protect private data. $\mathcal{C}$ cannot obtain information about the model parameters, and $\mathcal{S}$ cannot obtain $\mathcal{C}$'s input. The formal security definition based on the simulation paradigm [46], [47] is presented in Appendix I. Like prior work [4], [6], [8], [12], [13], Swift does not hide neural network structure (number of layers, kernel size, etc). Swift assumes that the communication channel between $\mathcal{C}$ and $\mathcal{S}$ is secure. In particular, the channel cannot be tampered with.

B. Protocols for Non-Linear Functions

At the beginning of the FHE-based secure neural network inference, $\mathcal{C}$ creates a CKKS key pair, encrypts the input with

Fig. 2. Swift ReLU protocol Π_{ReLU} .

the public key. Next, $\mathcal{C}$ sends the encrypted input $\text{Enc}(X)$ to $\mathcal{S}$. $\mathcal{S}$ employs the pre-trained model to perform inference. During the process, the input of a non-linear function is encrypted data. We take the first neural network layer as the example, after receiving $\text{Enc}(X)$ from $\mathcal{C}$, $\mathcal{S}$ uses weight parameters W to calculate linear function through homomorphic operations $\text{Enc}(Z) = W \boxtimes \text{Enc}(X)$, wherein $Z = X \cdot W$. $\text{Enc}(Z)$ is the input of the following non-linear function.

If the following non-linear function is ReLU, Swift compares each element x in Z with 0, represented as $\text{Enc}(x) \rightarrow \text{Enc}(\text{Max}(x, 0))$. The comparison in max pooling is different from ReLU. Max pooling performs comparison between elements in Z . Suppose Z contains elements (x, y) , max pooling selects $\text{Max}(x, y)$, represented as $\text{Enc}(x, y) \rightarrow \text{Enc}(\text{Max}(x, y))$. Next, we introduce the intuition for FHE-based ReLU and max pooling, then present the details of our proposed protocols.

1) *Intuition:* Since comparison between ciphertexts is time-consuming [31], Swift converts comparison between ciphertexts into comparison between plaintexts. During calculating ReLU, Swift needs to guarantee that private x is not recovered by $\mathcal{C}$ and $\mathcal{S}$. To achieve this goal, $\mathcal{C}$ randomly and uniformly generates r to hide x . Using a privacy-preserving method constructed by integrating FHE with SS, $\mathcal{S}$ can obtain plain $r \cdot x$, but cannot learn information about r and x . Next, $\mathcal{S}$ compares $r \cdot x$ with 0. By encrypting $\text{Max}(r \cdot x, 0)$ and several homomorphic operations for removing r , $\mathcal{S}$ effectively calculates the output $\text{Enc}(\text{Max}(x, 0))$. The intuition of FHE-based comparison for max pooling is similar with ReLU.

2) *Swift ReLU Protocol:* Figure 2 describes Swift protocol Π_{ReLU} for realizing ReLU functionality $\mathcal{F}_{\text{ReLU}}$. The input is

the result of the previous linear function $\text{Enc}(x)$, the output is $\text{Enc}(\text{ReLU}(x)) = \text{Enc}(\text{Max}(x, 0))$.

Operations independent with the input are performed in preprocess phase to improve performance. In preprocess phase, $\mathcal{C}$ randomly and uniformly generates encoded $r, r' \in \mathbb{R}_q$, the decoded $r \cdot r' = 1$. $\mathcal{C}$ calculates $\text{Enc}(r)$, $\text{Enc}(\text{Min}(0, r'))$ and $\text{Enc}(\text{Max}(r', -r'))$, sends them to $\mathcal{S}$.

In online phase, $\mathcal{S}$ performs homomorphic multiplication to calculate $\text{Enc}(r \cdot x)$. $\mathcal{C}$ holds the private key. $\mathcal{S}$ needs to interact with $\mathcal{C}$ to obtain plain $r \cdot x$, but does not disclose $r \cdot x$ to $\mathcal{C}$. Specifically, $\mathcal{S}$ randomly and uniformly generates $a \in \mathbb{R}_q$ that is a share of $r \cdot x$. $\mathcal{S}$ calculates another encrypted share $\text{Enc}(r \cdot x - a)$, sends $\text{Enc}(r \cdot x - a)$ to $\mathcal{C}$. $\mathcal{C}$ uses the private key to decrypt $\text{Enc}(r \cdot x - a)$, returns plain $(r \cdot x - a) \in \mathbb{R}_q$ to $\mathcal{S}$. $\mathcal{S}$ recovers $r \cdot x$ with two shares, $r \cdot x = (r \cdot x - a) + a \in \mathbb{R}_q$. Finally, $\mathcal{S}$ calculates the output with the equation: $\text{Enc}(\text{Max}(x, 0)) = (r \cdot x) \boxtimes \text{Enc}(\text{Min}(0, r')) \boxplus \text{Max}(r \cdot x, 0) \boxtimes \text{Enc}(\text{Max}(r', -r'))$.

In order to get the correct output, the products after multiplication (e.g. $r \cdot x$) in the protocol Π_{ReLU} cannot overflow CKKS coefficient modulus. In our implementation, the 0-level modulus is about 2^{60} , the encoded $x, r, r' \in \mathbb{R}_q$ can be taken no longer than 20 bits.

Theorem 1: The protocol Π_{ReLU} in Figure 2 securely realizes functionality $\mathcal{F}_{\text{ReLU}}$ in the presence of a semi-honest adversary controlling one corrupted party.

Proof: We prove that the online step (3) in Π_{ReLU} correctly computes $\text{Enc}(\text{Max}(x, 0))$. Due to the space limit, we only prove the case here where $x > 0$ and $r > 0$. The full version of the correctness proof is presented in Appendix II.

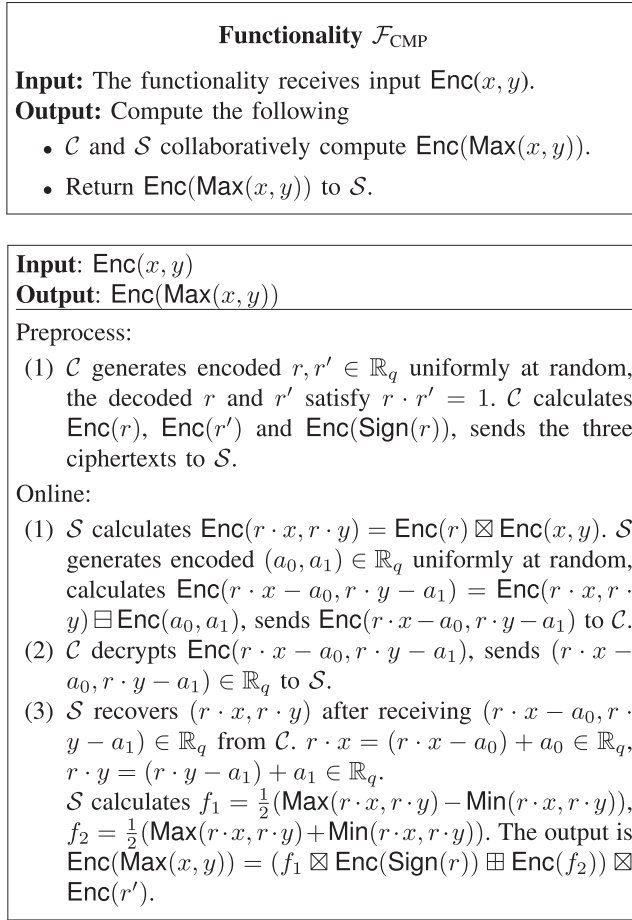
If $x > 0$ and $r > 0$, $\text{Enc}(\text{Max}(x, 0)) = \text{Enc}(x)$

$$\begin{aligned}
 & (r \cdot x) \boxtimes \text{Enc}(\text{Min}(0, r')) \boxplus \text{Max}(r \cdot x, 0) \boxtimes \text{Enc}(\text{Max}(r', -r')) \\
 &= (r \cdot x) \boxtimes \text{Enc}(0) \boxplus (r \cdot x) \boxtimes \text{Enc}(r') \\
 &= \text{Enc}(0) \boxplus \text{Enc}((r \cdot r') \cdot x) \\
 &= \text{Enc}(x) = \text{Enc}(\text{Max}(x, 0))
 \end{aligned}$$

Security. Π_{ReLU} tolerates one corrupted party. If $\mathcal{C}$ is corrupted, the adversary obtains r and $(r \cdot x - a) \in \mathbb{R}_q$. Since a is randomly generated and is secret to $\mathcal{C}$, the adversary is unable to infer x . Therefore, the adversary cannot learn information about model parameters. If $\mathcal{S}$ is corrupted, the adversary only obtains $r \cdot x \in \mathbb{R}_q$. Since r is randomly generated and is secret to $\mathcal{S}$, the adversary is unable to obtain $\mathcal{C}$'s private input according to [48] and [49]. Next, we present the formal security proof of Π_{ReLU} based on the simulation paradigm.

1) $\mathcal{C}$ is corrupted, $\mathcal{S}$ is honest. In this scenario, $\mathcal{C}$'s view includes the random r in preprocess and the received message $\text{Enc}(r \cdot x - a)$ in online step (1). The *simulator program* Sim constructs the view as follows:

- In the real interaction, r is randomly and uniformly generated. To construct the indistinguishable view, Sim also randomly and uniformly generates $\tilde{r}$ in the ideal interaction.
- Sim randomly and uniformly generates $\tilde{a}$ and calculates $\text{Enc}(\tilde{a})$. Due to the semantic security of RLWE [50], $\text{Enc}(\tilde{a})$ is indistinguishable from the received message $\text{Enc}(r \cdot x - a)$.

Fig. 3. Swift comparison protocol Π_{CMP} for max pooling.

- 2) $\mathcal{S}$ is corrupted, $\mathcal{C}$ is honest. In this scenario, $\mathcal{S}$'s view includes the random a in online step (1), received messages $(r \cdot x - a) \in \mathbb{R}_q$ in online step (2), $\text{Enc}(r)$, $\text{Enc}(\text{Min}(0, r'))$, and $\text{Enc}(\text{Max}(r', -r'))$ in preprocess. Sim constructs the view as follows:

- In the real interaction, a is randomly and uniformly generated. To construct the indistinguishable view, Sim also randomly and uniformly generates $\tilde{a}$ in the ideal interaction.
- Since a in the real interaction and $\tilde{a}$ in the ideal interaction are randomly and uniformly generated, both $(r \cdot x - a) \in \mathbb{R}_q$ in the real interaction and $(\tilde{r} \cdot \tilde{x} - \tilde{a}) \in \mathbb{R}_q$ generated by Sim follow the uniform random distribution.
- Sim creates $\text{Enc}(\tilde{r})$, $\tilde{r}$ is randomly and uniformly generated. Due to the semantic security of RLWE, $\text{Enc}(\tilde{r})$ is indistinguishable from $\text{Enc}(r)$. The proofs of $\text{Enc}(\text{Min}(0, r'))$ and $\text{Enc}(\text{Max}(r', -r'))$ are similar.

3) *Swift Comparison Protocol for Max Pooling:* Figure 3 describes Swift comparison protocol Π_{CMP} for realizing the functionality $\mathcal{F}_{\text{CMP}}$. Given input $\text{Enc}(x, y)$, Π_{CMP} outputs $\text{Enc}(\text{Max}(x, y))$. In preprocess phase, $\mathcal{C}$ performs input-independent operations, including randomly and uniformly generating encoded $r, r' \in \mathbb{R}_q$, calculating and sending $\text{Enc}(r)$, $\text{Enc}(r')$, $\text{Enc}(\text{Sign}(r))$. $\text{Sign}(r)$ denotes the sign of r . If r is

positive, $\text{Sign}(r) = 1$; if r is negative, $\text{Sign}(r) = -1$. In online phase, $\mathcal{S}$ performs homomorphic multiplication to calculate $\text{Enc}(r \cdot x, r \cdot y)$. With the assistance of $\mathcal{C}$, $\mathcal{S}$ can obtain plain $(r \cdot x, r \cdot y)$, but does not disclose $(r \cdot x, r \cdot y)$ to $\mathcal{C}$. Next, $\mathcal{S}$ calculates $f_1 = \frac{1}{2}(\text{Max}(r \cdot x, r \cdot y) - \text{Min}(r \cdot x, r \cdot y))$ and $f_2 = \frac{1}{2}(\text{Max}(r \cdot x, r \cdot y) + \text{Min}(r \cdot x, r \cdot y))$. The final output is $\text{Enc}(\text{Max}(x, y)) = (f_1 \boxtimes \text{Enc}(\text{Sign}(r)) \boxplus \text{Enc}(f_2)) \boxtimes \text{Enc}(r')$. Similar with Π_{ReLU} , in order to get the correct output, Π_{CMP} requires the products after multiplication (e.g. $r \cdot x, r \cdot y$) are less than the CKKS coefficient modulus we adopted.

Theorem 2: The protocol Π_{CMP} in Figure 3 securely realizes functionality $\mathcal{F}_{\text{CMP}}$ for max pooling in the presence of a semi-honest adversary controlling one corrupted party.

Proof: We prove that the online step (3) in Π_{CMP} correctly calculates the output $\text{Enc}(\text{Max}(x, y))$. Due to the space limit, we only prove the case here where $x > y$ and $r > 0$. The full version of the correctness proof is presented in Appendix III. The security analysis of Theorem 2 is similar with Theorem 1. We do not repeat here.

If $x > y$ and $r > 0$, $\text{Enc}(\text{Max}(x, y)) = \text{Enc}(x)$

$$\begin{aligned}
 & (f_1 \boxtimes \text{Enc}(\text{Sign}(r)) \boxplus \text{Enc}(f_2)) \boxtimes \text{Enc}(r') \\
 &= (f_1 \boxtimes \text{Enc}(1) \boxplus \text{Enc}(f_2)) \boxtimes \text{Enc}(r') \\
 &= \text{Enc}(f_1 + f_2) \boxtimes \text{Enc}(r') \\
 &= \text{Enc}(\text{Max}(r \cdot x, r \cdot y)) \boxtimes \text{Enc}(r') \\
 &= \text{Enc}(r \cdot x) \boxtimes \text{Enc}(r') \\
 &= \text{Enc}(x) = \text{Enc}(\text{Max}(x, y))
 \end{aligned}$$

C. Swift Secure 2PC-NN Inference

Neural networks consist of stacked layers. The output of the current layer is the input of the next layer. Each layer contains a linear function and a non-linear function. In this section, we present how to complete neural network inference with Swift protocols.

1) *Overall Design:* We present the overall design of Swift in Figure 4, and compare Swift with other secure inference frameworks. Taking a neural network layer as the example, we describe the process of calculating a linear function and the following non-linear function (e.g. ReLU).

In Swift framework, $\mathcal{S}$ holds the input $\text{Enc}(X)$ that is the computation result of the previous layer. $\mathcal{S}$ locally performs linear functions, free of interaction with $\mathcal{C}$. Specifically, $\mathcal{S}$ calculates $\text{Enc}(Z) = W \boxtimes \text{Enc}(X)$, wherein $Z = X \cdot W$, W is the weight parameters. $\text{Enc}(Z)$ is the input of the following non-linear ReLU. Using Swift protocol Π_{ReLU} , $\mathcal{S}$ obtains the output of the current layer, $\text{Enc}(\text{ReLU}(Z))$. $\text{Enc}(\text{ReLU}(Z))$ is the input of the next layer.

Current 2PC-NN inference schemes can be mainly classified into two categories. Type-I schemes employ hybrid cryptographic primitives: HE and SS for linear functions; GC and OT for non-linear functions. Cheetah is a typical representative of such schemes. We present the Cheetah framework in Figure 4b, and briefly introduce its computation process. When performing the linear function, $\mathcal{C}$ and $\mathcal{S}$ hold different shares of the input X . $\mathcal{C}$ holds X_0 and $\mathcal{S}$ holds X_1 , $X = X_0 + X_1$. $\mathcal{C}$ encrypts X_0 , sends $\text{Enc}(X_0)$ to $\mathcal{S}$. $\mathcal{S}$ calculates $\text{Enc}(Z) = W \boxtimes (\text{Enc}(X_0) \boxplus \text{Enc}(X_1))$, wherein $Z = X \cdot W$.

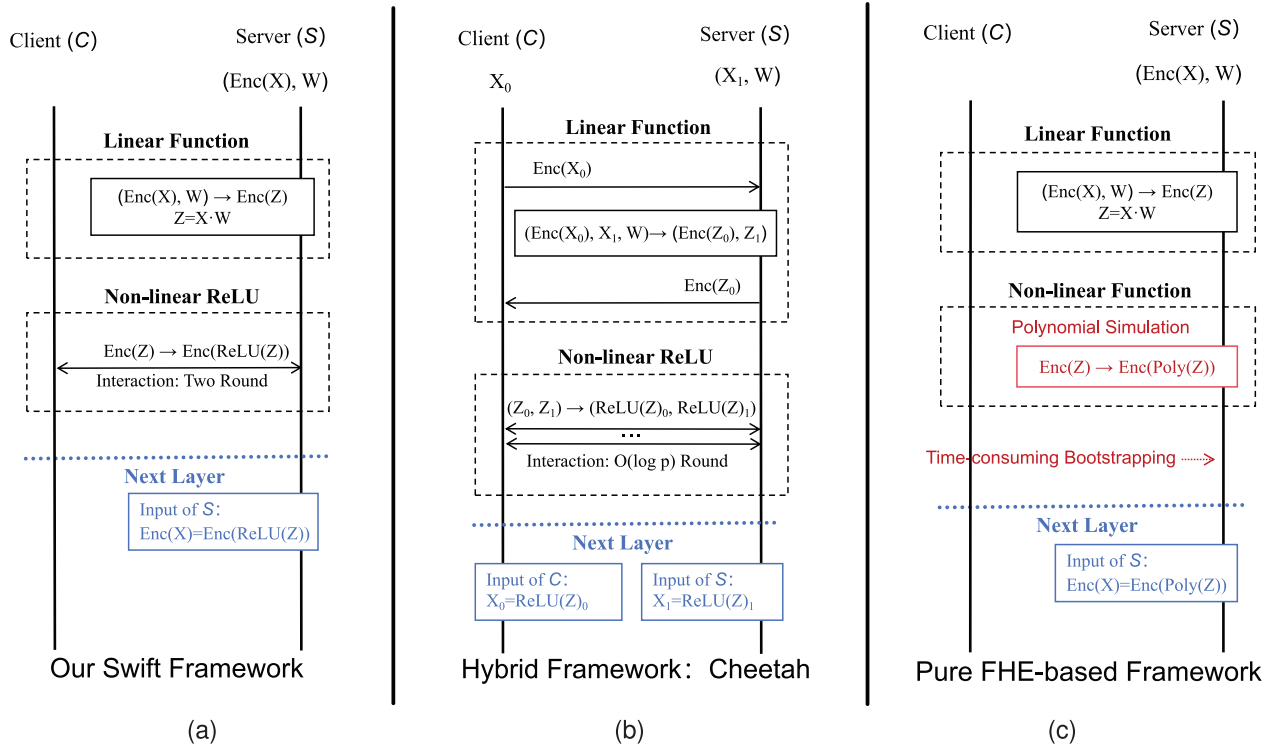


Fig. 4. Different secure 2PC-NN inference frameworks. (a) Our swift framework. (b) Hybrid framework cheetah. (c) Pure FHE-based framework.

$\mathcal{S}$ samples random Z_1 that is the share of Z , and calculates another encrypted share $\text{Enc}(Z_0) = \text{Enc}(Z) \boxplus \text{Enc}(Z_1)$. $\mathcal{S}$ sends $\text{Enc}(Z_0)$ to $\mathcal{C}$. $\mathcal{C}$ decrypts $\text{Enc}(Z_0)$ to obtain plain Z_0 . Therefore, $\mathcal{C}$ and $\mathcal{S}$ hold different shares of the output Z . $\mathcal{C}$ holds Z_0 and $\mathcal{S}$ holds Z_1 , which are the input of the following non-linear function ReLU. When performing ReLU, $\mathcal{C}$ and $\mathcal{S}$ collaboratively compute $\text{ReLU}(Z)$ with Cheetah's OT-based protocols [13]. $\mathcal{C}$ and $\mathcal{S}$ obtain different shares of $\text{ReLU}(Z)$. $\mathcal{C}$ holds $\text{ReLU}(Z)_0$ and $\mathcal{S}$ holds $\text{ReLU}(Z)_1$, which are the input of the next layer.

Compared with Cheetah, Swift has some special merits. Swift does not involve interaction between $\mathcal{C}$ and $\mathcal{S}$ when completing linear functions, also avoids encryption and decryption operations. As for non-linear functions, Swift designs more efficient protocols than Cheetah's OT-based protocols. OT-based protocols [12], [13] require $O(\log p)$ -round interactions for a p -bit integer. Swift non-linear protocols only need two-round interactions. The detailed performance comparison is presented in Section V-C.

Type-II secure inference schemes employ FHE to handle both linear functions and non-linear functions. We present the pure FHE-based framework in Figure 4c. The merit of Type-II schemes is that the entire inference is completed locally. However, since word-wise FHE schemes cannot directly calculate non-linear functions, researchers replace ReLU with approximate polynomials [32], [41], [51], then compute the encrypted polynomial $\text{Enc}(\text{Poly}(Z))$ by homomorphic operations. In addition, pure FHE-based schemes for deep neural networks (e.g. ResNet-20) [41], [51] require large encryption parameters and bootstrapping [44]. After a limited number of homomorphic operations, bootstrapping is necessary to

refresh the noise of ciphertexts. However, bootstrapping is an extremely time-consuming operation.

Compared with pure FHE-based framework, Swift accurately evaluates ReLU instead of using approximate polynomials. Further more, Swift can avoid time-consuming bootstrapping. Recall that our protocols Π_{ReLU} and Π_{CMP} have the *re-encryption after decryption* operation. It is an efficient manner to refresh ciphertexts (equivalent to bootstrapping function) [52], [53].

2) *Swift Linear Functions*: For linear functions, Cheetah [13] employs coefficient encoding to perform efficient computation based on the observation: *the polynomial multiplication itself can be viewed as a batch of inner products if the coefficients are arranged properly*. Fully connection and convolution can be rewritten as a batch of inner products. Therefore, by appropriately arranging elements from input and weight parameters onto polynomial coefficients and encrypting polynomials, homomorphic multiplication (corresponding to polynomial multiplication in plaintexts) directly obtains the encrypted result of fully connection or convolution. The detailed computation is presented in Section III-B and [13]. As shown in Figure 1b, the results of a linear function are sparsely arranged in specific coefficients of ciphertexts. To make full use of the free coefficients, BumbleBee [54] packs several “sparse” ciphertexts into a “dense” ciphertext. BumbleBee extracts the meaningful coefficients representing the results from multiple RLWE ciphertexts, then combines them into a single RLWE ciphertext through homomorphic automorphism.

Swift also employs coefficient encoding to encode input $\text{Enc}(X)$ and weight parameters W , then calculates fully connection or convolution $\text{Enc}(Z) = W \boxtimes \text{Enc}(X)$. Swift utilizes

the same technique with [54] to transform the sparsely-arranged result $\text{Enc}(Z)$ into the densely-arranged form.

Next, we present how to connect a linear function with the following non-linear function (e.g. ReLU). After a linear function, Swift obtains $\text{Enc}(Z)$ that is the input of the following ReLU. Suppose Z contains n_0 elements $\{z_0, \dots, z_{n_0-1}\}$. Swift employs the protocol Π_{ReLU} to calculate ReLU, n_0 different r are generated to hide n_0 elements in Z . However, as shown in Section III-B, only SIMD encoding (but not coefficient encoding) supports element-wise multiplication, represented as:

$$\begin{aligned} \text{Enc}(Z) \boxtimes \text{Enc}(R) \\ &= \text{Enc}(z_0, \dots, z_{n_0-1}) \boxtimes \text{Enc}(r_0, \dots, r_{n_0-1}) \\ &= \text{Enc}(z_0 \cdot r_0, \dots, z_{n_0-1} \cdot r_{n_0-1}) \end{aligned}$$

Therefore, The encoding method of $\text{Enc}(Z)$ needs to be changed before performing ReLU, from coefficient encoding to SIMD encoding.

We note that $\mathcal{C}$ cannot generate the same r to hide different elements in Z . In protocol Π_{ReLU} , if $\mathcal{S}$ is corrupted, the adversary can obtain $\{z_0 \cdot r, \dots, z_{n_0-1} \cdot r\}$ if using the same r . Since the sign of r only has two options (positive or negative), the sign of total n_0 elements in Z also have only two options. For example, the adversary: (1) guesses r is positive, the sign of each z_i in Z is same with exposed $z_i \cdot r$, (2) guesses r is negative, the sign of each z_i is opposite with exposed $z_i \cdot r$. However, if the sign of the input in a non-linear function is disclosed, the adversary can infer original client's input [48]. To prevent the adversary inferring the sign, Swift needs to generate different r . Actually, Swift randomly and uniformly generates $r_i \in \mathbb{R}_q$ for each z_i .

3) *Encoding Conversion*: For simplicity, this section defines $(\cdot)_{\text{Ccoeff}}$ and $(\cdot)_{\text{SIMD}}$, which represent coefficient encoding and SIMD encoding. $\text{Enc}((\cdot)_{\text{Ccoeff}})$ and $\text{Enc}((\cdot)_{\text{SIMD}})$ represent the coefficient-encoded ciphertext and SIMD-encoded ciphertext, respectively. Figure 5 describes our Swift encoding conversion protocol Π_{EC} for realizing the functionality $\mathcal{F}_{\text{EC}}$.

Given input $\text{Enc}(Z_{\text{Ccoeff}})$, Π_{EC} outputs $\text{Enc}(Z_{\text{SIMD}})$. In preprocess phase, $\mathcal{C}$ performs input-independent operations. $\mathcal{C}$ randomly and uniformly generates $B_{\text{Ccoeff}}, B_{\text{SIMD}} \in \mathbb{R}_q$. Both B_{Ccoeff} and B_{SIMD} encode the same n_0 -dimensional vector $\{b_0, b_1, \dots, b_{n_0-1}\}$, but employ different encoding method (coefficient method and SIMD method respectively). Next, $\mathcal{C}$ calculates $\text{Enc}(B_{\text{Ccoeff}})$ and $\text{Enc}(B_{\text{SIMD}})$, sends the two ciphertexts to $\mathcal{S}$.

In online phase, $\mathcal{S}$ performs homomorphic addition to calculate $\text{Enc}(Z_{\text{Ccoeff}} + B_{\text{Ccoeff}})$. With the assistance of $\mathcal{C}$, $\mathcal{S}$ can obtain plain $Z_{\text{Ccoeff}} + B_{\text{Ccoeff}}$, but does not disclose it to $\mathcal{C}$. Next, $\mathcal{S}$ changes the encoding method of $Z_{\text{Ccoeff}} + B_{\text{Ccoeff}}$, from coefficient encoding to SIMD encoding. $\mathcal{S}$ decodes $Z_{\text{Ccoeff}} + B_{\text{Ccoeff}}$ to obtain $\{z_0 + b_0, z_1 + b_1, \dots, z_{n_0-1} + b_{n_0-1}\}$, then re-encodes the vector with SIMD method to obtain $Z_{\text{SIMD}} + B_{\text{SIMD}}$. Finally, $\mathcal{S}$ calculates the output $\text{Enc}(Z_{\text{SIMD}}) = \text{Enc}(Z_{\text{SIMD}} + B_{\text{SIMD}}) \boxminus \text{Enc}(B_{\text{SIMD}})$.

Theorem 3: The protocol Π_{EC} in Figure 5 securely realizes functionality $\mathcal{F}_{\text{EC}}$ in the presence of a semi-honest adversary controlling one corrupted party.

Functionality $\mathcal{F}_{\text{EC}}$

Input: Coefficient-encoded ciphertext $\text{Enc}(Z_{\text{Ccoeff}})$.

Output: Compute the following

- $\mathcal{C}$ and $\mathcal{S}$ collaboratively compute SIMD-encoded ciphertext $\text{Enc}(Z_{\text{SIMD}})$.
- Return $\text{Enc}(Z_{\text{SIMD}})$ to $\mathcal{S}$.

Input: Coefficient-encoded ciphertext $\text{Enc}(Z_{\text{Ccoeff}})$

Output: SIMD-encoded ciphertext $\text{Enc}(Z_{\text{SIMD}})$

Preprocess:

- (1) $\mathcal{C}$ generates $B_{\text{Ccoeff}}, B_{\text{SIMD}} \in \mathbb{R}_q$, B_{Ccoeff} and B_{SIMD} encode the randomly sampled n_0 elements $\{b_0, b_1, \dots, b_{n_0-1}\}$ with coefficient and SIMD method respectively. $\mathcal{C}$ encrypts $B_{\text{Ccoeff}}, B_{\text{SIMD}}$ to obtain $\text{Enc}(B_{\text{Ccoeff}})$ and $\text{Enc}(B_{\text{SIMD}})$. $\mathcal{C}$ sends the two ciphertexts to $\mathcal{S}$.

Online:

- (1) $\mathcal{S}$ first calculates $\text{Enc}(Z_{\text{Ccoeff}} + B_{\text{Ccoeff}}) = \text{Enc}(Z_{\text{Ccoeff}}) \boxplus \text{Enc}(B_{\text{Ccoeff}})$. Then, $\mathcal{S}$ generates $A_{\text{Ccoeff}} \in \mathbb{R}_q$, A_{Ccoeff} encodes the randomly sampled n_0 elements $\{a_0, a_1, \dots, a_{n_0-1}\}$ with coefficient method. Next, $\mathcal{S}$ calculates $\text{Enc}(Z_{\text{Ccoeff}} + B_{\text{Ccoeff}} - A_{\text{Ccoeff}}) = \text{Enc}(Z_{\text{Ccoeff}} + B_{\text{Ccoeff}}) \boxminus \text{Enc}(A_{\text{Ccoeff}})$, sends $\text{Enc}(Z_{\text{Ccoeff}} + B_{\text{Ccoeff}} - A_{\text{Ccoeff}})$ to $\mathcal{C}$.
- (2) $\mathcal{C}$ decrypts $\text{Enc}(Z_{\text{Ccoeff}} + B_{\text{Ccoeff}} - A_{\text{Ccoeff}})$, sends $(Z_{\text{Ccoeff}} + B_{\text{Ccoeff}} - A_{\text{Ccoeff}}) \in \mathbb{R}_q$ to $\mathcal{S}$.
- (3) $\mathcal{S}$ recovers $Z_{\text{Ccoeff}} + B_{\text{Ccoeff}} = (Z_{\text{Ccoeff}} + B_{\text{Ccoeff}} - A_{\text{Ccoeff}}) + A_{\text{Ccoeff}} \in \mathbb{R}_q$. $\mathcal{S}$ changes the encoding method of $Z_{\text{Ccoeff}} + B_{\text{Ccoeff}}$, from coefficient encoding to SIMD encoding, obtaining $Z_{\text{SIMD}} + B_{\text{SIMD}}$. $\mathcal{S}$ encrypts $Z_{\text{SIMD}} + B_{\text{SIMD}}$ to obtain $\text{Enc}(Z_{\text{SIMD}} + B_{\text{SIMD}})$. Finally, $\mathcal{S}$ calculates the output $\text{Enc}(Z_{\text{SIMD}}) = \text{Enc}(Z_{\text{SIMD}} + B_{\text{SIMD}}) \boxminus \text{Enc}(B_{\text{SIMD}})$.

Fig. 5. Encoding conversion protocol Π_{EC} .

The essence of Π_{EC} is a *re-encryption after decryption* operation. Π_{EC} changes the encoding method when re-encrypting the plain data. Therefore, the correctness of Π_{EC} is obvious. Next, we analyze the security. If $\mathcal{C}$ is corrupted, the adversary obtains B_{Ccoeff} and $(Z_{\text{Ccoeff}} + B_{\text{Ccoeff}} - A_{\text{Ccoeff}}) \in \mathbb{R}_q$. Since $A_{\text{Ccoeff}} \in \mathbb{R}_q$ is randomly generated and is secret to $\mathcal{C}$, the adversary is unable to infer private Z_{Ccoeff} . Therefore, the adversary cannot learn information about model parameters. If $\mathcal{S}$ is corrupted, the adversary obtains $(Z_{\text{Ccoeff}} + B_{\text{Ccoeff}}) \in \mathbb{R}_q$. Since $B_{\text{Ccoeff}} \in \mathbb{R}_q$ is randomly generated and is secret to $\mathcal{S}$, the adversary is unable to infer Z_{Ccoeff} . Similarly, the adversary is unable to infer Z_{SIMD} because only $Z_{\text{SIMD}} + B_{\text{SIMD}}$ is obtained, and B_{SIMD} is secret to $\mathcal{S}$. Therefore, the adversary cannot learn information about $\mathcal{C}$'s input. Due to the space limit, the security proof based on the simulation paradigm is presented in Appendix IV.

4) *ReLU*: Swift employs our protocol Π_{ReLU} to securely calculate ReLU. The input of Π_{ReLU} is SIMD-encoded ciphertext $\text{Enc}(Z) = \text{Enc}(z_0, \dots, z_{n_0-1})$, the output of Π_{ReLU} is SIMD-encoded ciphertext $\text{Enc}(\text{ReLU}(Z)) = \text{Enc}(\text{Max}(z_0, 0), \dots, \text{Max}(z_{n_0-1}, 0))$. Since the next linear function employs

Input: $\text{Enc}(Z) = \text{Enc}(z_0, \dots, z_{n_2-1})$ with n_2 elements

Output: $\text{Enc}(\text{Max}(z_0, \dots, z_{n_2-1}))$

Preprocess:

- If n_2 is even:
 - (1) $\mathcal{C}$ generates encoded $B \in \mathbb{R}_q$ to hide Z . B contains n_2 elements $\{b_0, b_0, b_1, b_1, \dots, b_{\frac{n_2}{2}-1}, b_{\frac{n_2}{2}-1}\}$. $\mathcal{C}$ generates encoded $B' \in \mathbb{R}_q$, B' contains $\frac{n_2}{2}$ elements $\{b_0, b_1, \dots, b_{\frac{n_2}{2}-1}\}$. $\mathcal{C}$ calculates $\text{Enc}(B)$ and $\text{Enc}(B')$, then sends the two ciphertexts to $\mathcal{S}$.
- If n_2 is odd:
 - (1) $\mathcal{C}$ generates encoded $B \in \mathbb{R}_q$ to hide Z . B contains n_2 elements $\{b_0, b_0, b_1, b_1, \dots, b_{\lceil \frac{n_2}{2} \rceil - 2}, b_{\lceil \frac{n_2}{2} \rceil - 2}, b_{\lceil \frac{n_2}{2} \rceil - 1}\}$. $\mathcal{C}$ generates encoded $B' \in \mathbb{R}_q$, B' contains $\lceil \frac{n_2}{2} \rceil$ elements $\{b_0, b_1, \dots, b_{\lceil \frac{n_2}{2} \rceil - 2}, b_{\lceil \frac{n_2}{2} \rceil - 1}\}$. $\mathcal{C}$ calculates $\text{Enc}(B)$ and $\text{Enc}(B')$, then sends the two ciphertexts to $\mathcal{S}$.

Online:

- If n_2 is even:
 - (1) $\mathcal{S}$ calculates $\text{Enc}(Z + B) = \text{Enc}(Z) \boxplus \text{Enc}(B) = \text{Enc}(z_0 + b_0, z_1 + b_0, \dots, z_{n_2-2} + b_{\frac{n_2}{2}-1}, z_{n_2-1} + b_{\frac{n_2}{2}-1})$.
 - (2) Given input $\text{Enc}(Z + B)$, $\mathcal{C}$ and $\mathcal{S}$ perform Π_{CMP} in Figure 3 to calculate $\text{Enc}(\text{Max}(z_0 + b_0, z_1 + b_0), \dots, \text{Max}(z_{n_2-2} + b_{\frac{n_2}{2}-1}, z_{n_2-1} + b_{\frac{n_2}{2}-1}))$, which eliminates $\frac{n_2}{2}$ smaller elements.
 - (3) $\mathcal{S}$ calculates $\text{Enc}(\text{Max}(z_0, z_1), \dots, \text{Max}(z_{n_2-2}, z_{n_2-1})) = \text{Enc}(\text{Max}(z_0 + b_0, z_1 + b_0), \dots, \text{Max}(z_{n_2-2} + b_{\frac{n_2}{2}-1}, z_{n_2-1} + b_{\frac{n_2}{2}-1})) \boxminus \text{Enc}(B')$.
 - (4) Set $n'_2 = \frac{n_2}{2}$. If $n'_2 \neq 1$, performing Π_{MP} in Figure 6 again with new input $\text{Enc}(\text{Max}(z_0, z_1), \dots, \text{Max}(z_{n_2-2}, z_{n_2-1}))$ and n'_2 (n'_2 is the new n_2 in the next invocation of Π_{MP}). If $n'_2 = 1$, the protocol ends.
- If n_2 is odd:
 - (1) $\mathcal{S}$ calculates $\text{Enc}(Z + B) = \text{Enc}(Z) \boxplus \text{Enc}(B) = \text{Enc}(z_0 + b_0, z_1 + b_0, \dots, z_{n_2-3} + b_{\lceil \frac{n_2}{2} \rceil - 2}, z_{n_2-2} + b_{\lceil \frac{n_2}{2} \rceil - 2}, z_{n_2-1} + b_{\lceil \frac{n_2}{2} \rceil - 1})$.
 - (2) Given input $\text{Enc}(Z + B)$, $\mathcal{C}$ and $\mathcal{S}$ perform Π_{CMP} in Figure 3 to calculate $\text{Enc}(\text{Max}(z_0 + b_0, z_1 + b_0), \dots, \text{Max}(z_{n_2-3} + b_{\lceil \frac{n_2}{2} \rceil - 2}, z_{n_2-2} + b_{\lceil \frac{n_2}{2} \rceil - 2}, z_{n_2-1} + b_{\lceil \frac{n_2}{2} \rceil - 1}))$, which eliminates $\lfloor \frac{n_2}{2} \rfloor$ smaller elements.
 - (3) $\mathcal{S}$ calculates $\text{Enc}(\text{Max}(z_0, z_1), \dots, \text{Max}(z_{n_2-3}, z_{n_2-2}, z_{n_2-1})) = \text{Enc}(\text{Max}(z_0 + b_0, z_1 + b_0), \dots, \text{Max}(z_{n_2-3} + b_{\lceil \frac{n_2}{2} \rceil - 2}, z_{n_2-2} + b_{\lceil \frac{n_2}{2} \rceil - 2}, z_{n_2-1} + b_{\lceil \frac{n_2}{2} \rceil - 1})) \boxminus \text{Enc}(B')$.
 - (4) Set $n'_2 = \lceil \frac{n_2}{2} \rceil$. Performing Π_{MP} in Figure 6 again with new input $\text{Enc}(\text{Max}(z_0, z_1), \dots, \text{Max}(z_{n_2-3}, z_{n_2-2}, z_{n_2-1}))$ and n'_2 (n'_2 is the new n_2 in the next invocation of Π_{MP}).

Fig. 6. Max pooling protocol Π_{MP} .

coefficient encoding, Swift converts SIMD-encoded output to the coefficient-encoded form after calculating ReLU.

5) *Max Pooling*: If the window size of max pooling is $n_1 \times n_1$, Swift selects the maximum element from $n_2 = n_1^2$ encrypted elements, represented as $\text{Enc}(z_0, \dots, z_{n_2-1}) \rightarrow \text{Enc}(\text{Max}(z_0, \dots, z_{n_2-1}))$. Π_{MP} in Figure 6 describes Swift max pooling protocol. We note that the security of Π_{MP} depends on our proposed Π_{CMP} . Additional operations in Π_{MP} aim to improve performance. We will explain later.

In the protocol Π_{MP} , $\lfloor \cdot \rfloor$ and $\lceil \cdot \rceil$ denote the flooring and ceiling functions. Π_{MP} contains $\lceil \log n_2 \rceil$ rounds. In each round, Π_{MP} compares every two adjacent elements in Z and eliminates the smaller one. Therefore, each round totally eliminates $\lfloor \frac{n_2}{2} \rfloor$ elements smaller than $\text{Max}(z_0, \dots, z_{n_2-1})$. If n_2 is even, the current round outputs $\text{Enc}(\text{Max}(z_0, z_1), \dots, \text{Max}(z_{n_2-2}, z_{n_2-1}))$, which is the input of the next round. Set $n'_2 = \frac{n_2}{2}$, n'_2 is the new n_2 in the next round. If n_2 is odd, the output is $\text{Enc}(\text{Max}(z_0, z_1), \dots, \text{Max}(z_{n_2-3}, z_{n_2-2}, z_{n_2-1}))$, and n'_2 is $\lceil \frac{n_2}{2} \rceil$ (new n_2 in the next round). After $\lceil \log n_2 \rceil$ rounds, all the n_2 elements in Z are compared, the protocol outputs the final result $\text{Enc}(\text{Max}(z_0, \dots, z_{n_2-1}))$.

Taking even n_2 as the example, we explain the details in each round. Π_{MP} securely compares elements in Z by invoking Π_{CMP} . Before performing Π_{CMP} , $\mathcal{C}$ generates encoded $B, B' \in \mathbb{R}_q$. B is used for hiding private Z , and contains

n_2 elements $\{b_0, b_0, b_1, b_1, \dots, b_{\frac{n_2}{2}-1}, b_{\frac{n_2}{2}-1}\}$. B' contains $n_2/2$ elements $\{b_0, b_1, \dots, b_{\frac{n_2}{2}-1}\}$. Each b_i ($i \in [0, \frac{n_2}{2} - 1]$) is randomly and uniformly sampled, but the $2i$ -th and $(2i + 1)$ -th elements in $\{b_0, b_0, b_1, b_1, \dots, b_{\frac{n_2}{2}-1}, b_{\frac{n_2}{2}-1}\}$ correspond to the same b_i . $\mathcal{C}$ calculates $\text{Enc}(B)$ and $\text{Enc}(B')$. Through homomorphic addition, $\mathcal{S}$ obtains $\text{Enc}(Z + B) = \text{Enc}(z_0 + b_0, \dots, z_{n_2-1} + b_{\frac{n_2}{2}-1})$. $\text{Enc}(Z + B)$ is the input of Π_{CMP} . Next, $\mathcal{C}$ and $\mathcal{S}$ perform Π_{CMP} to calculate $\text{Enc}(\text{Max}(z_0 + b_0, z_1 + b_0), \dots, \text{Max}(z_{n_2-2} + b_{\frac{n_2}{2}-1}, z_{n_2-1} + b_{\frac{n_2}{2}-1}))$. Finally, by homomorphically subtracting $\text{Enc}(B')$, $\mathcal{S}$ obtains the result of the current round $\text{Enc}(\text{Max}(z_0, z_1), \dots, \text{Max}(z_{n_2-2}, z_{n_2-1}))$.

In Π_{CMP} , $\mathcal{C}$ can generate same r to hide different elements in $Z + B$. As mentioned in [48], the sign of elements in Z should be private. If $\mathcal{S}$ is corrupted, the adversary can obtain each $r \cdot (z_i + b_i)$, $i \in [0, n_2 - 1]$. Since the sign of r only has two options (positive or negative), the sign of total n_2 elements in $Z + B$ also have only two options: (1) the sign of each $z_i + b_i$ in $Z + B$ is same with exposed $r \cdot (z_i + b_i)$ for positive r , (2) the sign of each $z_i + b_i$ is opposite with exposed $r \cdot (z_i + b_i)$ for negative r . We note that the secret b_i (for the adversary) is randomly generated, and can be positive or negative. Therefore, the adversary is unable to infer the sign of z_i even if the correct sign of $z_i + b_i$ is exposed. Similar with Π_{ReLU} , in order to get the correct output, Π_{MP} requires the

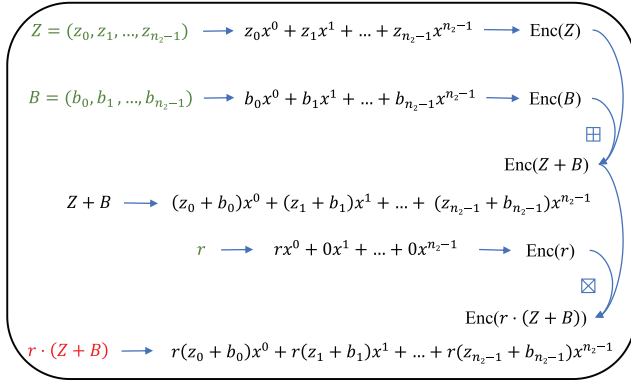


Fig. 7. Encoding method in max pooling.

products after multiplication (e.g. $r \cdot (z_i + b_i)$) are less than the CKKS coefficient modulus we adopted.

In Π_{MP} , secure comparison is performed based on Π_{CMP} . Other operations aim to hide private z_i by adding b_i . We present the detailed encoding method in Figure 7. Since coefficient encoding supports $Z + B$ (element-wise addition) and $r \cdot (Z+B)$ (multiplication of scalar r with every element in $Z+B$), we avoid encoding conversion before performing Π_{MP} , thereby reducing the online computation overhead. Recall that Swift calculates linear functions with coefficient encoding, followed by the non-linear functions (e.g. max pooling). Therefore, the input $\text{Enc}(Z)$ of Π_{MP} is a coefficient-encoded ciphertext. The element z_i in Z is placed on the i -th coefficient of the Z -encoded polynomial. Similarly, the element b_i in B is placed on the i -th coefficient of the B -encoded polynomial. After polynomial addition (corresponding to homomorphic addition), the i -th coefficient of $(Z+B)$ -encoded polynomial is $z_i + b_i$. Next, we place r on the constant term of the r -encoded polynomial, the other coefficients are placed with 0. Through polynomial multiplication (corresponding to homomorphic multiplication), the i -th coefficient is $r \cdot (z_i + b_i)$.

6) *Discussion*: Swift works for semi-honest security, which protects privacy when clients and servers follow protocols. However, a malicious client can launch model extraction attacks [55] to obtain the model parameters by initiating a series of inference queries and forging the secret shares during interactions. Resisting model extraction attacks is not within the scope of Swift. The cryptographic primitives proposed in [55] and [56] are supposed to be combined to achieve malicious security.

V EVALUATIONS

In this section, we first present the theoretical analysis of Swift protocols. Next, we evaluate the specific performance of Swift linear functions, non-linear functions, encoding conversion function and neural network inference. We compare Swift with Cheetah [13], one of the most efficient two-party secure inference schemes.

A. Experiment Setup

The implementation of Swift is build on microsoft SEAL library [43]. Our linear function references the open-source

TABLE I

ROUND AND COMMUNICATION OF SWIFT PROTOCOLS. IN THIS TABLE, COMP. IS THE ABBREVIATION OF COMPARISON

Protocol	Round	Communication	Commu. per Comp.
Π_{ReLU}	2	$3n \log q$ bits	$6 \log q$ bits
Π_{CMP}	2	$3n \log q$ bits	$6 \log q$ bits
Π_{EC}	2	$3n \log q$ bits	—

Cheetah [57], which uses the coefficient encoding to calculate convolution and fully connection. Swift source code is implemented in C++ and is compiled by gcc (version 11.2.0) on Ubuntu 20.04.

1) *Testbed Environment*: All benchmarks are performed in the server equipped with 2 16-core 3.2Ghz AMD EPYC CPUs and 128GB RAM. Each party performs the computation on a single thread and occupies one port. With Linux `tc` command, we consider two network settings: a LAN setting with a shared 10Gbps connection and approximately 0.3ms RTT latency and a WAN setting with a 100Mbps maximum throughput and 40ms RTT latency.

2) *Dataset and Neural Network*: We employ the widely used MNIST dataset containing of a collection of hand-written numbers. Each image is represented as 28×28 pixels. Our goal is to predict the corresponding number for each image. We employ the 3-layer convolutional neural network (CNN) derived from Chameleon [6]. The first layer is convolution followed by ReLU activation function. The parameters of the convolution are: kernel of size 5×5 , stride of 2, map count of 5. The second layer is fully connection followed by ReLU activation function. The fully connection outputs 100 nodes from incoming 720 nodes. The last layer contains a fully connection, which outputs 10 nodes from incoming 100 nodes.

B. Theoretical Analysis

1) *Analysis of Swift Protocols*: We first summarize the theoretical interaction round and communication cost of Swift protocols in Table I. Since preprocess operations are independent with input and can be executed in advance, we only consider the online operations.

Both Π_{ReLU} and Π_{CMP} require two-round interactions. In Π_{ReLU} , $\mathcal{S}$ sends a ciphertext to $\mathcal{C}$ in online step (1), and $\mathcal{C}$ sends a plaintext to $\mathcal{S}$ in online step (2). Therefore, the communication cost is $2n \log q + n \log q = 3n \log q$, n is the degree and q is the coefficient modulus. Similarly, the communication cost of Π_{CMP} is $3n \log q$. Since Π_{ReLU} employs SIMD encoding, a ciphertext packs $\frac{n}{2}$ elements. One invocation of Π_{ReLU} performs $\frac{n}{2}$ comparisons (compare each element with 0). The communication cost of one comparison is $\frac{3n \log q}{n/2} = 6 \log q$. Π_{CMP} employs coefficient encoding, a ciphertext packs n elements. One invocation of Π_{CMP} performs $\frac{n}{2}$ comparisons (compare every two elements). The communication cost of one comparison is also $6 \log q$. We use the same manner to analyze Swift encoding conversion protocol Π_{EC} . Π_{EC} requires two rounds, and the communication cost is $3n \log q$. Since one invocation of Π_{EC} changes the encoding method of $\frac{n}{2}$ elements, the communication cost of converting one element is $6 \log q$.

TABLE II
ROUND AND COMMUNICATION COST COMPARISON BETWEEN CHEETAH,
HEFUN AND SWIFT

Method	Round	Communication
Cheetah [13]	$O(\log p)$	$<15p$ bits
HeFUN [40]	2	$8 \log q'$ bits, $q' > q$
Swift	2	$6 \log q$ bits

2) *Comparison With Prior Art*: In order to evaluate the non-linear function ReLU, prior works have constructed secure comparison protocols using OT, GC, and random masking. We compare interaction round and communication cost between Swift and prior works.

- **Swift vs. OT-based-CMP**: Cheetah [13] and CryptFlow2 [12] employ OT to construct Millionaire protocol, further evaluate ReLU. Compared with CryptFlow2, Cheetah utilizes VOLE-style OT extension to replace IKNP-style OT extension to improve efficiency. As shown in Table II, OT-based ReLU requires $O(\log p)$ interaction rounds for a p -bit integer. In contrast, Swift only requires two rounds. The communication cost of Swift is at the same level as Cheetah. If OT-based Millionaire protocol is employed to calculate ReLU, Cheetah transmits about $15p$ bits. Swift transmits $6 \log q$ bits when using CKKS.
- **Swift vs. GC-based-CMP**: ReLU based on GC [24], [58] requires two interaction rounds (the same as Swift), but the communication cost far exceeds Swift. Gazelle [7] employs FHE to calculate linear functions, then transforms FHE ciphertext to the input of GC, finally executes GC-based ReLU. The online communication cost of ReLU-1000 in Gazelle is 1.68MB. In contrast, Swift only consumes 0.05MB (represented in Table III). MP2ML [59] calculates ReLU with ABY framework [60], which contains both GC and GMW [61] (Goldreich-Micali-Wigderson) protocols. Compared with GC-based ReLU, GMW-based ReLU has increased interaction rounds, but the communication cost is reduced. ReLU-1000 with GC and GMW in MP2ML consume 2.0MB and 0.9MB respectively, one order of magnitude higher than Swift's communication cost.
- **Swift vs. Random-Masking-HeFUN**: Both Swift and HeFUN [40] mask private data with random numbers, but they utilize different methods to calculate ReLU. Swift has less communication cost and less multiplicative depth than HeFUN.

Communication cost: For a fair comparison, we suppose Swift and HeFUN employ the same FHE scheme (e.g. CKKS). We use q and q' to denote the coefficient modulus in Swift and HeFUN respectively. The communication complexity of Swift and HeFUN is presented in Table II. HeFUN is the FHE-based method. The communication cost of ReLU is two ciphertexts, consuming $2 \cdot 2n \log q' = 4n \log q'$. Since one ciphertext packs $n/2$ elements, the communication of one comparison is $\frac{4n \log q'}{n/2} = 8 \log q'$. Swift's ReLU is the FHE-SS hybrid method. The communication cost is a ciphertext and a share (plaintext), consuming $2n \log q + n \log q = 3n \log q$. The communication of one comparison is $\frac{3n \log q}{n/2} = 6 \log q$. Next,

we illustrate that $q' > q$ because HeFUN requires larger multiplicative depth.

Multiplicative depth: The depth requirement of HeFUN's ReLU is 2. The first depth is consumed when calculating $\text{Enc}(\text{Sign}(x))$, and the second depth is consumed when performing $\text{Enc}(\text{ReLU}(x)) = \frac{1}{2} \boxtimes (\text{Enc}(x) \boxplus \text{Enc}(x) \boxtimes \text{Enc}(\text{Sign}(x)))$. Since the next linear function consumes additional 1 depth, HeFUN requires total 3 depth. In our Swift, Π_{ReLU} directly calculates ReLU with the equation $\text{Enc}(\text{ReLU}(x)) = (r \cdot x) \boxtimes \text{Enc}(\text{Min}(0, r')) \boxplus \text{Max}(r \cdot x, 0) \boxtimes \text{Enc}(\text{Max}(r', -r'))$, consuming only 1 depth. The next linear function consumes additional 1 depth. Therefore, Swift requires total 2 depth, less than HeFUN.

C. Benchmarks

We set CKKS parameters in SEAL library as follows: the degree n of the polynomial modulus ($x^n + 1$) is 4096, the CKKS modulus is selected to satisfy 128-bit security $q \approx 2^{60} \times 2^{49}$. We note that the low-level 60 bits are used for ciphertexts. Another 49 bits in the high level are used for key projects.

1) *Non-Linear Functions and Encoding Conversion*: We present the specific performance of our proposed non-linear protocols in Table III. The end-to-end time and communication are accumulated for 2^{10} comparisons in ReLU and max pooling. The windows size of max pooling is (2×2) .

Cheetah is evaluated based on the open source code [57]. We compare the performance of Swift non-linear protocols with Cheetah's OT-based non-linear protocols, the result is presented in Table IV. The end-to-end time refers to online execution time because preprocess operations can be executed in advance. Compared with Cheetah, ReLU improves 7.2× under the LAN setting and 7.4× under the WAN setting, respectively. Max pooling improves 4.5× under the LAN setting and 13.3× under the WAN setting.

The performance of our encoding conversion protocol is presented in Table III. The batch size is 2^{10} . The result illustrates that Swift only takes about 29ms to complete the conversion between a coefficient-encoded ciphertext and a SIMD-encoded ciphertext under the WAN setting. Previous work NeuJeans [39] performs encoding conversion through (inverse) discrete Fourier transform. Although NeuJeans can locally complete the conversion, the performance overhead is huge. The coefficient-to-SIMD conversion is combined with bootstrapping. The cost is in seconds. In contrast, Swift Π_{EC} achieves the same functionality with obvious performance improvement. Our cost is in milliseconds.

2) *Linear Functions*: We evaluate Swift linear functions, and compare them with Cheetah. The result is presented in Table V. The parameters (HW, Ker, s, c_i, c_o) of convolution represent the input size, kernel size, stride, input channel and output channel. The parameters (n_i, n_o) of fully connection represent the length of input vector and output vector. The size of weight matrix is $n_o \cdot n_i$.

We first analyze the end-to-end time under the WAN setting. The performance of Swift has obvious improvement. The reason is that convolution and fully connection in Swift do not involve communication. S locally computes linear functions. However, Cheetah is the SS-based framework, requires

TABLE III

END-TO-END TIME AND COMMUNICATION EVALUATION FOR SWIFT NON-LINEAR FUNCTIONS AND ENCODING CONVERSION FUNCTION

Benchmark	LAN		WAN		Communication (MB)	
	End2End Time (ms)		End2End Time (ms)			
	Prep.	Online	Prep.	Online	Prep.	Online
ReLU	1.53	1.32	22.39	34.46	0.10	0.05
Max pooling	1.98	3.16	14.90	58.25	0.16	0.05
Encoding conversion	1.23	1.36	19.47	29.09	0.07	0.05

TABLE IV

END-TO-END TIME COMPARISON BETWEEN SWIFT AND CHEETAH NON-LINEAR FUNCTIONS

Benchmark	Method	End2End Time (ms)	
		LAN	WAN
ReLU	Swift	1.32	34.46
	Cheetah [13]	9.57	256.30
	Speedup	7.2×	7.4×
Max pooling	Swift	3.16	58.25
	Cheetah [13]	14.38	778.32
	Speedup	4.5×	13.3×

TABLE V

END-TO-END TIME COMPARISON BETWEEN SWIFT AND CHEETAH LINEAR FUNCTIONS. IN THIS TABLE, CONV DENOTES CONVOLUTION FUNCTION AND FC DENOTES FULLY CONNECTION FUNCTION

Benchmark	Parameters	Method	End2End Time (ms)	
			LAN	WAN
Conv	$(HW, Ker, s, c_i, c_o) = (28 \cdot 28, 5 \cdot 5, 2, 1, 5)$	Swift	1.12	1.12
		Cheetah [13]	6.67	75.68
		Speedup	5.9×	67.5×
FC	$(n_i, n_o) = (100, 10)$	Swift	0.43	0.43
		Cheetah [13]	2.92	59.54
		Speedup	6.7×	138.4×
FC	$(n_i, n_o) = (720, 100)$	Swift	4.07	4.07
		Cheetah [13]	17.86	95.75
		Speedup	4.3×	23.5×

TABLE VI

END-TO-END TIME EVALUATION FOR SWIFT SECURE NEURAL NETWORK INFERENCE

Benchmark	LAN		WAN	
	Prep.	Online	Prep.	Online
Secure Inference	16.44	24.14	111.37	379.78

TABLE VII

END-TO-END TIME COMPARISON BETWEEN SWIFT AND CHEETAH SECURE NEURAL NETWORK INFERENCE

Benchmark	Method	End2End Time (ms)	
		LAN	WAN
Secure Inference	Swift	24.14	379.78
	Cheetah [13]	42.88	728.87
	Speedup	1.7×	1.9×

interaction between $\mathcal{C}$ and $\mathcal{S}$. Under the WAN setting, data transmission consumes a significant amount of time.

Table V also illustrates that the performance of Swift improves more than 4× under the LAN setting. The result is reasonable. As shown in Section IV-C1, compared with Swift, Cheetah requires additional operations, including encrypting the input share, sending the encrypted input/output share, and decrypting the output share. Therefore, Cheetah spends more execution time than Swift.

3) *Secure Neural Network Inference*: We present the performance of Swift secure neural network inference in Table VI. Table VII compares Swift with Cheetah, the end-to-end time refers to online execution time. Compared with Cheetah, Swift improves 1.7× in the LAN setting and 1.9× in the WAN setting, respectively.

The performance improvement of secure inference is not as significant as independent non-linear protocols because the scale of neural network for MNIST dataset is small. When evaluating Π_{ReLU} , Swift packs $4096/2 = 2048$ elements into one ciphertext. Therefore, one execution of Π_{ReLU} can perform 2048 comparisons in parallel because of the SIMD character. However, the number of nodes in each neural network layer is less than 2048. For example, we only need to calculate ReLU-100 in the second layer. Lots of free slots are wasted. We plan to implement secure inference for large-scale neural networks using our Swift framework in the future.

VI CONCLUSION

We present a fast secure 2PC-NN inference scheme, called Swift. Swift designs new ReLU and max pooling protocols by integrating FHE with SS. Our proposed non-linear protocols have less execution time compared with the State-of-the-Art. Swift also designs efficient encoding conversion protocol between a coefficient-encoded ciphertext and a SIMD-encoded ciphertext. When performing secure inference, Swift employs coefficient encoding to quickly calculate convolution and fully connection, and employs SIMD encoding for our proposed ReLU protocol after encoding conversion. The evaluation indicates that compared with Cheetah (USENIX 2022), the execution time of ReLU, max pooling, secure inference under a WAN setting improves 7.4×, 13.3×, 1.9×, respectively.

APPENDIX A THREAT MODEL AND SECURITY

Swift follows the simulation paradigm to prove security against the static semi-honest adversary $\mathcal{A}$. Specifically, there are two defined interactions: a real interaction and an ideal interaction. During a real interaction, client $\mathcal{C}$ and server $\mathcal{S}$ perform the protocol $\Pi_{\mathcal{F}}$ in the presence of $\mathcal{A}$. $\mathcal{A}$ can corrupt $\mathcal{C}$ or $\mathcal{S}$, and obtain the corresponding view $\text{View}_{\mathcal{A}}$. A party's view consists of the input, received messages and internal random tapes. During an ideal interaction, each party sends the input to a trusted party, the trusted party computes functionality $\mathcal{F}$ faithfully. We use Sim to denote a *simulator*

program in the ideal interaction. Given $\mathcal{A}$'s input and output, Sim can simulate $\mathcal{A}$'s view in the protocol $\Pi_{\mathcal{F}}$.

$$\text{Sim}(\text{in}, \text{out}) = \widetilde{\text{View}}_{\mathcal{A}}$$

To achieve semantic security, the simulated view should be computationally indistinguishable from the view of $\mathcal{A}$ in the real interaction.

Definition (Security). A protocol Π is said to securely realize a functionality $\mathcal{F}$ if there exists a simulator Sim , satisfying that for the party corrupted by $\mathcal{A}$, the view produced by Sim is computationally indistinguishable from the view in the real interaction.

$$\text{Sim}(\text{in}, \text{out}) \approx \text{View}_{\mathcal{A}}$$

APPENDIX B PROOF OF THEOREM 1

To prove Theorem 1, we need to illustrate online step (3) in Π_{ReLU} correctly computes $\text{Enc}(\text{Max}(x, 0))$. The prove is as follows.

Case 1: $x > 0$ and $r > 0$, $\text{Enc}(\text{Max}(x, 0)) = \text{Enc}(x)$

$$\begin{aligned} & (r \cdot x) \boxtimes \text{Enc}(\text{Min}(0, r')) \boxplus \text{Max}(r \cdot x, 0) \boxtimes \text{Enc}(\text{Max}(r', -r')) \\ &= (r \cdot x) \boxtimes \text{Enc}(0) \boxplus (r \cdot x) \boxtimes \text{Enc}(r') \\ &= \text{Enc}(0) \boxplus \text{Enc}((r \cdot r') \cdot x) \\ &= \text{Enc}(x) = \text{Enc}(\text{Max}(x, 0)) \end{aligned}$$

Case 2: $x > 0$ and $r < 0$, $\text{Enc}(\text{Max}(x, 0)) = \text{Enc}(x)$

$$\begin{aligned} & (r \cdot x) \boxtimes \text{Enc}(\text{Min}(0, r')) \boxplus \text{Max}(r \cdot x, 0) \boxtimes \text{Enc}(\text{Max}(r', -r')) \\ &= (r \cdot x) \boxtimes \text{Enc}(r') \boxplus 0 \boxtimes \text{Enc}(-r') \\ &= \text{Enc}((r \cdot r') \cdot x) \boxplus \text{Enc}(0) \\ &= \text{Enc}(x) = \text{Enc}(\text{Max}(x, 0)) \end{aligned}$$

Case 3: $x \leq 0$ and $r > 0$, $\text{Enc}(\text{Max}(x, 0)) = \text{Enc}(0)$

$$\begin{aligned} & (r \cdot x) \boxtimes \text{Enc}(\text{Min}(0, r')) \boxplus \text{Max}(r \cdot x, 0) \boxtimes \text{Enc}(\text{Max}(r', -r')) \\ &= (r \cdot x) \boxtimes \text{Enc}(0) \boxplus 0 \boxtimes \text{Enc}(r') \\ &= \text{Enc}(0) = \text{Enc}(\text{Max}(x, 0)) \end{aligned}$$

Case 4: $x \leq 0$ and $r < 0$, $\text{Enc}(\text{Max}(x, 0)) = \text{Enc}(0)$

$$\begin{aligned} & (r \cdot x) \boxtimes \text{Enc}(\text{Min}(0, r')) \boxplus \text{Max}(r \cdot x, 0) \boxtimes \text{Enc}(\text{Max}(r', -r')) \\ &= (r \cdot x) \boxtimes \text{Enc}(r') \boxplus (r \cdot x) \boxtimes \text{Enc}(-r') \\ &= \text{Enc}(r \cdot r' \cdot x) \boxplus \text{Enc}(r \cdot r' \cdot x) \\ &= \text{Enc}(0) = \text{Enc}(\text{Max}(x, 0)) \end{aligned}$$

APPENDIX C PROOF OF THEOREM 2

To prove Theorem 2, we need to illustrate online step (3) in Π_{CMP} correctly computes $\text{Enc}(\text{Max}(x, y))$. The prove is as follows.

Case 1: $x > y$ and $r > 0$, $\text{Enc}(\text{Max}(x, y)) = \text{Enc}(x)$

$$\begin{aligned} & (f_1 \boxtimes \text{Enc}(\text{Sign}(r)) \boxplus \text{Enc}(f_2)) \boxtimes \text{Enc}(r') \\ &= (f_1 \boxtimes \text{Enc}(1) \boxplus \text{Enc}(f_2)) \boxtimes \text{Enc}(r') \\ &= \text{Enc}(f_1 + f_2) \boxtimes \text{Enc}(r') \\ &= \text{Enc}(\text{Max}(r \cdot x, r \cdot y)) \boxtimes \text{Enc}(r') \end{aligned}$$

$$\begin{aligned} &= \text{Enc}(r \cdot x) \boxtimes \text{Enc}(r') \\ &= \text{Enc}(x) = \text{Enc}(\text{Max}(x, y)) \end{aligned}$$

Case 2: $x > y$ and $r < 0$, $\text{Enc}(\text{Max}(x, y)) = \text{Enc}(x)$

$$\begin{aligned} & (f_1 \boxtimes \text{Enc}(\text{Sign}(r)) \boxplus \text{Enc}(f_2)) \boxtimes \text{Enc}(r') \\ &= (f_1 \boxtimes \text{Enc}(-1) \boxplus \text{Enc}(f_2)) \boxtimes \text{Enc}(r') \\ &= \text{Enc}(f_2 - f_1) \boxtimes \text{Enc}(r') \\ &= \text{Enc}(\text{Min}(r \cdot x, r \cdot y)) \boxtimes \text{Enc}(r') \\ &= \text{Enc}(r \cdot x) \boxtimes \text{Enc}(r') \\ &= \text{Enc}(x) = \text{Enc}(\text{Max}(x, y)) \end{aligned}$$

Case 3: $x \leq y$ and $r > 0$, $\text{Enc}(\text{Max}(x, y)) = \text{Enc}(y)$

$$\begin{aligned} & (f_1 \boxtimes \text{Enc}(\text{Sign}(r)) \boxplus \text{Enc}(f_2)) \boxtimes \text{Enc}(r') \\ &= (f_1 \boxtimes \text{Enc}(1) \boxplus \text{Enc}(f_2)) \boxtimes \text{Enc}(r') \\ &= \text{Enc}(f_1 + f_2) \boxtimes \text{Enc}(r') \\ &= \text{Enc}(\text{Max}(r \cdot x, r \cdot y)) \boxtimes \text{Enc}(r') \\ &= \text{Enc}(r \cdot y) \boxtimes \text{Enc}(r') \\ &= \text{Enc}(y) = \text{Enc}(\text{Max}(x, y)) \end{aligned}$$

Case 4: $x \leq y$ and $r < 0$, $\text{Enc}(\text{Max}(x, y)) = \text{Enc}(y)$

$$\begin{aligned} & (f_1 \boxtimes \text{Enc}(\text{Sign}(r)) \boxplus \text{Enc}(f_2)) \boxtimes \text{Enc}(r') \\ &= (f_1 \boxtimes \text{Enc}(-1) \boxplus \text{Enc}(f_2)) \boxtimes \text{Enc}(r') \\ &= \text{Enc}(f_2 - f_1) \boxtimes \text{Enc}(r') \\ &= \text{Enc}(\text{Min}(r \cdot x, r \cdot y)) \boxtimes \text{Enc}(r') \\ &= \text{Enc}(r \cdot y) \boxtimes \text{Enc}(r') \\ &= \text{Enc}(y) = \text{Enc}(\text{Max}(x, y)) \end{aligned}$$

APPENDIX D SECURITY OF THEOREM 3

We present the formal security proof of Π_{EC} based on the simulation paradigm.

1) $\mathcal{C}$ is corrupted, $\mathcal{S}$ is honest. In this scenario, $\mathcal{C}$'s view includes the random $\mathbf{B}_{\text{Coeff}}$, $\mathbf{B}_{\text{SIMD}}$ in preprocess and the received message $\text{Enc}(\mathbf{Z}_{\text{Coeff}} + \mathbf{B}_{\text{Coeff}} - \mathbf{A}_{\text{Coeff}})$ in online step (1). The simulator program Sim constructs the view as follows:

- In the real interaction, $\mathbf{B}_{\text{Coeff}}$ and $\mathbf{B}_{\text{SIMD}}$ are randomly and uniformly generated. To construct the indistinguishable view, Sim generates $\widetilde{\mathbf{B}}_{\text{Coeff}}$, $\widetilde{\mathbf{B}}_{\text{SIMD}} \in \mathbb{R}_q$ in the ideal interaction. $\widetilde{\mathbf{B}}_{\text{Coeff}}$ and $\widetilde{\mathbf{B}}_{\text{SIMD}}$ encodes the randomly and uniformly sampled vector $\{\widetilde{b}_0, \widetilde{b}_1, \dots, \widetilde{b}_{n_0-1}\}$ with coefficient method and SIMD method respectively.
- Sim generates $\widetilde{\mathbf{R}}_{\text{Coeff}} \in \mathbb{R}_q$, $\widetilde{\mathbf{R}}_{\text{Coeff}}$ encodes the randomly and uniformly sampled vector $\{\widetilde{r}_0, \widetilde{r}_1, \dots, \widetilde{r}_{n_0-1}\}$ with coefficient method. Due to the semantic security of RLWE, $\text{Enc}(\widetilde{\mathbf{R}}_{\text{Coeff}})$ is indistinguishable from $\text{Enc}(\mathbf{Z}_{\text{Coeff}} + \mathbf{B}_{\text{Coeff}} - \mathbf{A}_{\text{Coeff}})$.

2) $\mathcal{S}$ is corrupted, $\mathcal{C}$ is honest. In this scenario, $\mathcal{S}$'s view includes the random $\mathbf{A}_{\text{Coeff}}$ in online step (1), received messages $(\mathbf{Z}_{\text{Coeff}} + \mathbf{B}_{\text{Coeff}} - \mathbf{A}_{\text{Coeff}}) \in \mathbb{R}_q$ in online step (2), $\text{Enc}(\mathbf{B}_{\text{Coeff}})$ and $\text{Enc}(\mathbf{B}_{\text{SIMD}})$ in preprocess. Sim constructs the view as follows:

- In the real interaction, A_{Coeff} is randomly and uniformly generated. To construct the indistinguishable view, Sim generates $\tilde{A}_{\text{Coeff}} \in \mathbb{R}_q$ in the ideal interaction. $\tilde{A}_{\text{Coeff}}$ encodes the randomly and uniformly sampled vector $\{\tilde{a}_0, \tilde{a}_1, \dots, \tilde{a}_{n_0-1}\}$ with coefficient method.
- In the real interaction, the encoded elements in $(Z_{\text{Coeff}} + B_{\text{Coeff}} - A_{\text{Coeff}}) \in \mathbb{R}_q$ follow the uniform random distribution. To construct the indistinguishable view, Sim generates $\tilde{R}_{\text{Coeff}} \in \mathbb{R}_q$, $\tilde{R}_{\text{Coeff}}$ encodes the randomly and uniformly sampled vector $\{\tilde{r}_0, \tilde{r}_1, \dots, \tilde{r}_{n_0-1}\}$ with coefficient method.
- Sim creates $\text{Enc}(\tilde{B}_{\text{Coeff}})$, $\tilde{B}_{\text{Coeff}}$ is randomly and uniformly generated. Due to the semantic security of RLWE, $\text{Enc}(\tilde{B}_{\text{Coeff}})$ is indistinguishable from $\text{Enc}(B_{\text{Coeff}})$. The proof of $\text{Enc}(B_{\text{SIMD}})$ is similar.

REFERENCES

- [1] E. Angelini, G. Di Tollo, and A. Roli, "A neural network approach for credit risk evaluation," *Quart. Rev. Econ. Finance*, vol. 48, no. 4, pp. 733–755, Nov. 2008.
- [2] R. Gilad-Bachrach, N. Dowlin, K. Laine, K. Lauter, M. Naehrig, and J. Wernsing, "CryptoNets: Applying neural networks to encrypted data with high throughput and accuracy," in *Proc. Int. Conf. Mach. Learn.*, 2016, pp. 201–210.
- [3] E. Hesamifard, H. Takabi, and M. Ghasemi, "CryptoDL: Towards deep learning over encrypted data," in *Proc. Annu. Comput. Secur. Appl. Conf.*, vol. 11, 2016, pp. 1–2.
- [4] J. Liu, M. Juuti, Y. Lu, and N. Asokan, "Oblivious neural network predictions via minion transformations," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2017, pp. 619–631.
- [5] P. Mohassel and Y. Zhang, "SecureML: A system for scalable privacy-preserving machine learning," in *Proc. IEEE Symp. Security Privacy (SP)*, May 2017, pp. 19–38.
- [6] M. S. Riazzi, C. Weinert, O. Tkachenko, E. M. Songhori, T. Schneider, and F. Koushanfar, "Chameleon: A hybrid secure computation framework for machine learning applications," in *Proc. Asia Conf. Comput. Commun. Security*, 2018, pp. 707–721.
- [7] C. Juvekar, V. Vaikuntanathan, and A. Chandrakasan, "GAZELLE: A low latency framework for secure neural network inference," in *Proc. 27th USENIX Secur. Symp. (USENIX Security)*, 2018, pp. 1651–1669.
- [8] W. Z. Srinivasan, P. Akshayaram, and P. R. Ada, "Delphi: A cryptographic inference service for neural networks," in *Proc. 29th USENIX Secur. Symp.*, 2019, pp. 2505–2522.
- [9] F. Boemer, Y. Lao, R. Cammarota, and C. Wierzynski, "NGraph-HE: A graph compiler for deep learning on homomorphically encrypted data," in *Proc. 16th ACM Int. Conf. Comput. Frontiers*, Apr. 2019, pp. 3–13.
- [10] F. Boemer, A. Costache, R. Cammarota, and C. Wierzynski, "NGraph-HE2: A high-throughput framework for neural network inference on encrypted data," in *Proc. ACM Workshop Encrypted Comput. Appl. Homomorphic Cryptogr.*, 2019, pp. 45–56.
- [11] A. Brutzkus, R. Gilad-Bachrach, and O. Elisha, "Low latency privacy preserving inference," in *Proc. Int. Conf. Mach. Learn.*, 2019, pp. 812–821.
- [12] D. Rathee et al., "CrypTFlow2: Practical 2-party secure inference," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, Oct. 2020, pp. 325–342.
- [13] Z. Huang, W. J. Lu, C. Hong, and J. Ding, "Cheetah: Lean and fast secure two-party deep neural network inference," in *Proc. 31st USENIX Secur. Symp.*, Aug. 2022, pp. 809–826.
- [14] B. D. Rouhani, M. S. Riazzi, and F. Koushanfar, "DeepSecure: Scalable provably-secure deep learning," in *Proc. 55th Annu. Design Autom. Conf.*, 2018, pp. 1–6.
- [15] M. S. Riazzi, M. Samragh, H. Chen, K. Laine, K. Lauter, and F. Koushanfar, "XONN: XNOR-based oblivious deep neural network inference," in *Proc. 28th USENIX Secur. Symp.*, Aug. 2019, pp. 1501–1518.
- [16] N. Kumar, M. Rathee, N. Chandran, D. Gupta, A. Rastogi, and R. Sharma, "CrypTFlow: Secure TensorFlow inference," in *Proc. IEEE Symp. Secur. Privacy (SP)*, May 2020, pp. 336–353.
- [17] P. Mohassel and P. Rindal, "ABY3: A mixed protocol framework for machine learning," in *Proc. ACM SIGSAC Conf. Comput. Commun. Security*, 2018, pp. 35–52.
- [18] A. Patra and A. Suresh, "BLAZE: Blazing fast privacy-preserving machine learning," in *Proc. 27th Netw. Distrib. Syst. Secur. Symp.*, 2020.
- [19] S. Wagh, D. Gupta, and N. Chandran, "SecureNN: 3-party secure computation for neural network training," *Proc. Privacy Enhancing Technol.*, vol. 2019, no. 3, pp. 26–49, Jul. 2019.
- [20] H. Chaudhari, R. Rachuri, and A. Suresh, "Trident: Efficient 4PC framework for privacy preserving machine learning," in *Proc. 27th Netw. Distrib. Syst. Secur. Symp.*, 2020.
- [21] Y. Li and W. Xu, "PrivPy: General and scalable privacy-preserving data mining," in *Proc. 25th ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining*, Jul. 2019, pp. 1299–1307.
- [22] F. Liu, X. Xie, and Y. Yu, "Scalable multi-party computation protocols for machine learning in the honest-majority setting," in *Proc. 33rd USENIX Secur. Symp.*, 2024, pp. 1939–1956.
- [23] N. P. Smart and F. Vercauteren, "Fully homomorphic SIMD operations," *Des., Codes Cryptography*, vol. 71, no. 1, pp. 57–81, Apr. 2014.
- [24] A. C. C. Yao, "How to generate and exchange secrets," in *Proc. 27th Annu. Symp. Found. Comput. Sci.*, Oct. 1986, pp. 162–167.
- [25] J. Fan and F. Vercauteren, "Somewhat practical fully homomorphic encryption," *Cryptol. ePrint Arch.*, Paper 2012/144, 2012. [Online]. Available: <https://eprint.iacr.org/2012/144>
- [26] Z. Brakerski, C. Gentry, and V. Vaikuntanathan, "(Leveled) fully homomorphic encryption without bootstrapping," *ACM Trans. Comput. Theory*, vol. 6, no. 3, pp. 1–36, Jul. 2014.
- [27] J. H. Cheon, A. Kim, M. Kim, and Y. Song, "Homomorphic encryption for arithmetic of approximate numbers," in *Proc. 23rd Int. Conf. Theory Appl. Cryptol. Inf. Secur.* Cham, Switzerland: Springer, 2017, pp. 409–437.
- [28] I. Chillotti, N. Gama, M. Georgieva, and M. Izabachène, "TFHE: Fast fully homomorphic encryption over the torus," *J. Cryptol.*, vol. 33, no. 1, pp. 34–91, Jan. 2020.
- [29] L. Ducas and D. Micciancio, "FHEW: Bootstrapping homomorphic encryption in less than a second," in *Proc. Annu. Int. Conf. Theory Appl. Cryptograph. Techn.* Berlin, Germany: Springer, 2015, pp. 617–640.
- [30] C. Boura, N. Gama, M. Georgieva, and D. Jetchev, "CHIMERA: Combining ring-LWE-based fully homomorphic encryption schemes," *J. Math. Cryptol.*, vol. 14, no. 1, pp. 316–338, 2020.
- [31] W.-J. Lu, Z. Huang, C. Hong, Y. Ma, and H. Qu, "PEGASUS: Bridging polynomial and non-polynomial evaluations in homomorphic encryption," in *Proc. IEEE Symp. Security Privacy (SP)*, May 2021, pp. 1057–1073.
- [32] D. Kim and C. Guyot, "Optimized privacy-preserving CNN inference with fully homomorphic encryption," *IEEE Trans. Inf. Forensics Security*, vol. 18, pp. 2175–2187, 2023.
- [33] V. Costan and S. Devadas, "Intel SGX explained," *Cryptol. ePrint Arch.*, Paper 2016/086, 2016. [Online]. Available: <https://eprint.iacr.org/2016/086>
- [34] J. V. Bulck, N. Weichbrodt, R. Kapitza, F. Piessens, and R. Strackx, "Telling your secrets without page faults: Stealthy page table-based attacks on enclaved execution," in *Proc. 26th USENIX Secur. Symp.*, Aug. 2017, pp. 1041–1056.
- [35] Y. Xu, W. Cui, and M. Peinado, "Controlled-channel attacks: Deterministic side channels for untrusted operating systems," in *Proc. IEEE Symp. Secur. Privacy*, May 2015, pp. 640–656.
- [36] A. Moghimi, G. Irazoqui, and T. Eisenbarth, "CacheZoom: How SGX amplifies the power of cache attacks," in *Proc. 19th Int. Conf.*, Taipei, Taiwan. Cham, Switzerland: Springer, Sep. 2017, pp. 69–90.
- [37] J. V. Bulck et al., "Foreshadow: Extracting the keys to the Intel SGX kingdom with transient out-of-order execution," in *Proc. 27th USENIX Secur. Symp.*, 2018, pp. 991–1008.
- [38] H. Ragab, A. Milburn, K. Razavi, H. Bos, and C. Giuffrida, "CrossTalk: Speculative data leaks across cores are real," in *Proc. IEEE Symp. Secur. Privacy (SP)*, May 2021, pp. 1852–1867.
- [39] J. Hyung Ju et al., "NeuJeans: Private neural network inference with joint optimization of convolution and FHE bootstrapping," 2023, *arXiv:2312.04356*.
- [40] D. T. K. Nguyen, D. H. Duong, W. Susilo, Y.-W. Chow, and T. A. Ta, "HeFUN: Homomorphic encryption for unconstrained secure neural network inference," *Future Internet*, vol. 15, no. 12, p. 407, Dec. 2023.
- [41] E. Lee et al., "Low-complexity deep convolutional neural networks on fully homomorphic encryption using multiplexed parallel convolutions," in *Proc. Int. Conf. Mach. Learn.*, 2022, pp. 12403–12422.
- [42] J. H. Cheon, K. Han, A. Kim, M. Kim, and Y. Song, "A full RNS variant of approximate homomorphic encryption," in *Selected Areas in Cryptography—SAC*. Cham, Switzerland: Springer, 2018, pp. 347–368.
- [43] (2021). *Microsoft SEAL (Release 3.7)*. Microsoft Research, Redmond, WA, USA. [Online]. Available: <https://github.com/Microsoft/SEAL>

- [44] C. Gentry, "Fully homomorphic encryption using ideal lattices," in *Proc. 41st Annu. ACM Symp. Theory Comput. (STOC)*, 2009, pp. 169–178.
- [45] M. J. Flynn, "Some computer organizations and their effectiveness," *IEEE Trans. Comput.*, vol. C-21, no. 9, pp. 948–960, Sep. 1972.
- [46] Y. Lindell, "How to simulate it—A tutorial on the simulation proof technique," in *Tutorials on the Foundations of Cryptography*. Cham, Switzerland: Springer, 2017, pp. 277–346.
- [47] D. Evans, V. Kolesnikov, and M. Rosulek, "A pragmatic introduction to secure multi-party computation," *Found. Trends Privacy Secur.*, vol. 2, nos. 2–3, pp. 70–246, 2018.
- [48] M. Wei, W. Zhu, L. Cui, X. Li, and Q. Li, "Privacy leakage in privacy-preserving neural network inference," in *Proc. Eur. Symp. Res. Comput. Secur.* Cham, Switzerland: Springer, 2022, pp. 133–152.
- [49] L. Shen, X. Chen, J. Shi, Y. Dong, and B. Fang, "An efficient 3-party framework for privacy-preserving neural network inference," in *Proc. Eur. Symp. Res. Comput. Secur.* Cham, Switzerland: Springer, 2020, pp. 419–439.
- [50] M. Albrecht et al., "Homomorphic encryption security standard," HomomorphicEncryption.org, Toronto, ON, Canada, Tech. Rep., Nov. 2018.
- [51] J.-W. Lee et al., "Privacy-preserving machine learning with fully homomorphic encryption for deep neural network," *IEEE Access*, vol. 10, pp. 30039–30054, 2022.
- [52] W. Wang et al., "Toward scalable fully homomorphic encryption through light trusted computing assistance," 2019, *arXiv:1905.07766*.
- [53] S. Sav et al., "POSEIDON: Privacy-preserving federated neural network learning," in *Proc. Netw. Distrib. Syst. Secur. Symp.*, 2021.
- [54] W.-J. Lu et al., "BumbleBee: Secure two-party inference framework for large transformers," *Cryptol. ePrint Arch.*, Paper 2023/1678, 2023. [Online]. Available: <https://eprint.iacr.org/2023/1678>
- [55] R. Lehmkuhl, P. Mishra, A. Srinivasan, and R. A. Popa, "Muse: Secure inference resilient to malicious clients," in *Proc. 30th USENIX Secur. Symp.*, 2021, pp. 2201–2218.
- [56] N. Chandran, D. Gupta, S. L. B. Obbattu, and A. Shah, "SIMC: ML inference secure against malicious clients at semi-honest cost," in *Proc. 31st USENIX Secur. Symp.*, 2022, pp. 1361–1378.
- [57] (2022). *OpenCheetah*. [Online]. Available: <https://github.com/Alibaba-Gemini-Lab/OpenCheetah>
- [58] X. Wang, A. J. Malozemoff, and J. Katz. (2016). *EMP-Toolkit: Efficient Multiparty Computation Toolkit*. [Online]. Available: <https://github.com/emp-toolkit>
- [59] F. Boemer, R. Cammarota, D. Demmler, T. Schneider, and H. Yalame, "MP2ML: A mixed-protocol machine learning framework for private inference," in *Proc. 15th Int. Conf. Availability, Rel. Secur.*, 2020, pp. 1–10.
- [60] D. Demmler, T. Schneider, and M. Zohner, "ABY—A framework for efficient mixed-protocol secure two-party computation," in *Proc. Netw. Distrib. Syst. Secur. Symp.*, 2015.
- [61] O. Goldreich, S. Micali, and A. Wigderson, "How to play any mental game," in *Proc. 19th Annu. ACM Symp. Theory Comput.*, New York, NY, USA, 1987, pp. 218–229.



Yu Fu received the M.S. degree from the University of Chinese Academy of Sciences in 2022. He is currently pursuing the Ph.D. degree with the School of Cyber Science and Technology, University of Science and Technology of China. His research interests include applied cryptography and privacy-preserving machine learning.



Yu Tong received the B.E. degree from the University of Science and Technology of China in 2023, where he is currently pursuing the M.S. degree with the School of Cyber Science and Technology. His research interests include privacy-preserving machine learning.



Yijing Ning received the B.E. degree from the University of Science and Technology of China in 2023, where he is currently pursuing the M.S. degree with the School of Cyber Science and Technology. His research interests include cryptographic engineering and high-performance computing.



Tianshi Xu received the bachelor's degree from Beihang University. He is currently pursuing the Ph.D. degree with the School of Integrated Circuit, Peking University. He is currently filled with great passion for AI safety and privacy, especially privacy-preserving deep learning (PPDL).



Meng Li (Member, IEEE) received the bachelor's degree from Peking University, Beijing, China, in 2013, and the Ph.D. degree from The University of Texas at Austin, Austin, TX, USA, in 2018. He is currently a Tenure-Track Assistant Professor with Peking University, jointly affiliated with the Institute for Artificial Intelligence and the School of Integrated Circuits. Before joining Peking University, he was a Staff Research Scientist and the Tech Lead with the Facebook On-Device AI Team, with a focus on research and productization of efficient AI algorithms and hardware/systems for next-generation AR/VR devices. He has published more than 80 articles. His research interests include efficient and secure multimodal AI acceleration algorithms and hardware. He received two Best Paper Awards from HOST 2017 and GLSVLSI 2018. He also receives the CCF Integrated Circuits Early Career Award, EDAA Outstanding Dissertation Award, and First Place in ACM Student Research Competition Grand Final (Graduate Category).



Jingqiang Lin (Senior Member, IEEE) received the M.S. and Ph.D. degrees from the University of Chinese Academy of Sciences in 2004 and 2009, respectively. He is currently a Full Professor with the School of Cyber Security, University of Science and Technology of China. His research interests include applied cryptography and system security.



Dengguo Feng received the bachelor's degree from Shaanxi Normal University, Xi'an, China, in 1988, and the M.S. and Ph.D. degrees from Xidian University, Xi'an, in 1993 and 1995, respectively. He is currently a Professor with the Institute of Software, Chinese Academy of Sciences, Beijing, China. His research interests include trusted computing and information assurance. He has authored or co-authored more than 200 publications in international journals and conferences and presided over the research work and project formulation of more than 20 international and national standards in the field of information security technology. He has won the First Prize of National Award for Progress in Science and Technology and the Second Prize of National Award for Technological Invention.